QCM de révision : Deep Learning

Trois QCM de 40 questions : facile, moyen, difficile

MSA-DATB09-03 · Neural Networks & Deep Learning Foundations · Guillaume Saint-Cirgue

Consignes. Une seule bonne réponse par question. Note tes réponses dans la grille au début de chaque QCM, puis compare avec le corrigé commenté à la fin du QCM.

Durée conseillée. 30 minutes pour le QCM facile, 45 minutes pour le moyen, 60 minutes pour le difficile. Pour mémoire, l'examen dure 90 minutes, sans IA : lecture et écriture de code PyTorch, formules des 8 sessions, et questions de réflexion (expliquer, diagnostiquer, choisir, justifier).

Vocabulaire. Les termes techniques restent en anglais, comme dans le cours : log loss, autograd, weight decay, dropout, batch norm, skip connection, cell state...

Tags. Chaque question indique sa session (S2 à S8) et sa notion : si tu te trompes, tu sais quelle partie du cours revoir.

Hors programme ici. Le perceptron (session 1) et l'initialisation Xavier / He ne sont pas traités.

Notions couvertes

Session 2	Sigmoid neuron, log loss, gradient (a - y)x, gradient descent, NumPy puis PyTorch, BCEWithLogitsLoss
Session 3	Réseau de neurones (couches, formes, forward pass), XOR, non-linéarité, backpropagation à la main, autograd
Session 4	PyTorch en détail : tenseurs, nn.Linear, nn.Sequential, losses, optimiseurs, DataLoader, boucle d'entraînement, eval()
Session 5	ReLU et autres activations, dérivées, vanishing gradient, dead ReLU
Session 6	Overfitting et diagnosis, weight decay, dropout, early stopping, batch norm, momentum, Adam, gradient clipping
Session 7	Convolution, padding, stride, taille de sortie, paramètres, pooling, weight sharing, ResNet, transfer learning
Session 8	RNN, backpropagation through time, LSTM, attention, sentence embeddings, production checklist, synthèse du cours

QCM 1 : niveau facile

Définitions, formules de base, lecture de schémas et petits calculs directs.

Grille de réponses

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**

Question 1 S2 · Sigmoid neuron

Pour le neurone sigmoïde $a = \sigma(z) = 1/(1 + e^{-z})$, que vaut $\sigma(0)$, et entre quelles valeurs $\sigma(z)$ reste-t-elle ?

- **A.** $\sigma(0) = 0$ et $-1 < \sigma(z) < 1$
- **B.** $\sigma(0) = 1$ et $0 < \sigma(z) < 1$
- **C.** $\sigma(0) = 0.5$ et $\sigma(z) > 0$, sans maximum
- **D.** $\sigma(0) = 0.5$ et $0 < \sigma(z) < 1$

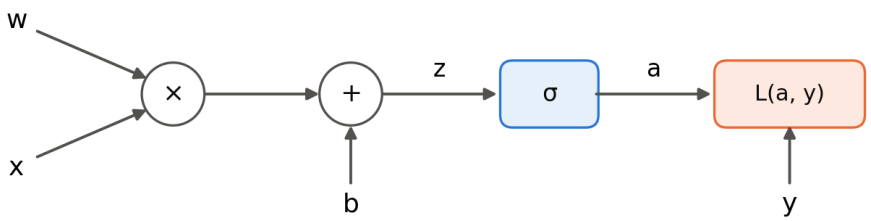
Question 2 S2 · Log loss

Loss d'un exemple : $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$. Pour un exemple de classe $y = 1$, $\mathcal{L}$ vaut :

- **A.** $-\log(1 - a)$
- **B.** $-\log a$
- **C.** $(a - 1)^2$
- **D.** $\log a$

Question 3 S2 · Gradient

Sur ce graphe de calcul (neurone sigmoïde + log loss), la chain rule donne $\partial\mathcal{L}/\partial w = \partial\mathcal{L}/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$. Que vaut ce gradient ?



- **A.** $(a - y)\, a(1 - a)\, x$
- **B.** $(a - y)\, x$
- **C.** $(y - a) / x$
- **D.** $2(a - y)\, x$

Question 4 S2 · Gradient descent

Quelle est la règle de mise à jour du gradient descent ?

- **A.** $w \leftarrow w + \eta\, \partial J/\partial w$
- **B.** $w \leftarrow \eta\, \partial J/\partial w$
- **C.** $w \leftarrow w - \partial J/\partial \eta$
- **D.** $w \leftarrow w - \eta\, \partial J/\partial w$

Question 5 S2 · Decision boundary

Le neurone sigmoïde prédit la classe 1 quand $a \geq 0.5$. À quoi cela correspond-il ?

- **A.** $z \geq 0.5$: la frontière devient une courbe qui suit les données
- **B.** $z \geq 1$: la frontière devient un cercle autour des points
- **C.** $a \geq 0.5$ ne dépend pas du score z
- **D.** $z = w \cdot x + b \geq 0$: la frontière reste une droite

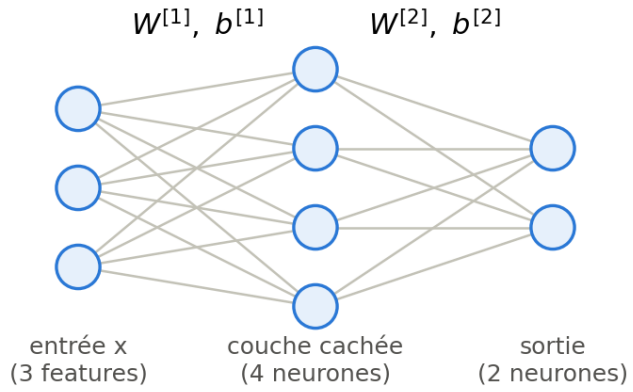
Question 6 S3 · XOR

Le réseau XOR du cours combine trois neurones. Quelle décomposition utilise-t-il ?

- **A.** AND(OR(x_1, x_2), NAND(x_1, x_2))
- **B.** OR(AND(x_1, x_2), NAND(x_1, x_2))
- **C.** NAND(OR(x_1, x_2), AND(x_1, x_2))
- **D.** AND(OR(x_1, x_2), AND(x_1, x_2))

Question 7 S3 · Shapes

Dans ce réseau 3-4-2, quelle est la forme de $W^{[1]}$?



- **A.** (3, 4)
- **B.** (4, 3)
- **C.** (4, 1)
- **D.** (3, 2)

Question 8 S3 · Non-linearity

Que se passe-t-il si l'on oublie l'activation entre deux couches `nn.Linear` ?

- **A.** Le code plante dès le premier forward
- **B.** Le réseau devient deux fois plus expressif
- **C.** Le gradient devient nul pour tous les poids
- **D.** Le modèle reste linéaire : une seule droite

Question 9 S3 · Backpropagation

Pendant l'apprentissage d'un réseau, dans quel sens circulent les valeurs et les gradients ?

- **A.** Valeurs vers la sortie, gradients vers l'entrée
- **B.** Valeurs et gradients de l'entrée vers la sortie
- **C.** Gradients vers la sortie, valeurs vers l'entrée
- **D.** Les gradients sont calculés dans un ordre aléatoire

Question 10 S3 · Autograd

Que signifie `requires_grad=True` sur un tenseur PyTorch ?

- **A.** PyTorch enregistre ses opérations pour calculer son gradient
- **B.** Le tenseur est gelé : l'optimiseur ne pourra plus jamais le modifier
- **C.** Le tenseur est automatiquement copié sur le GPU
- **D.** Son gradient est remis à zéro après chaque opération

Question 11 S4 · Training loop

Quel ordre des étapes est correct dans la boucle d'entraînement PyTorch ?

- **A.** forward, loss, `step()`, `backward()`, `zero_grad()`
- **B.** `backward()`, forward, loss, `zero_grad()`, `step()`
- **C.** `zero_grad()`, forward, loss, `backward()`, `step()`
- **D.** forward, `backward()`, loss, `step()`, `zero_grad()`

Question 12 S4 · Losses

Avec `nn.CrossEntropyLoss`, que doit renvoyer la dernière couche du modèle ?

- **A.** Des probabilités, après une softmax
- **B.** Des valeurs entre 0 et 1, après une sigmoïde
- **C.** Un vecteur one-hot de la classe prédite
- **D.** Des logits bruts, sans softmax

Question 13 S4 · Tensors

Pourquoi écrire `X = train_set.data.float() / 255` pour FashionMNIST ?

- **A.** Pour passer de float32 à des entiers uint8 entre 0 et 255
- **B.** Pour transformer chaque image en un vecteur one-hot de 784 valeurs
- **C.** Pour mélanger les images avant de créer le DataLoader
- **D.** Pour passer d'entiers uint8 (0 à 255) à des float32 entre 0 et 1

Question 14 S4 · DataLoader

50 000 images d'entraînement et `DataLoader(..., batch_size=128, shuffle=True)`. Combien de mises à jour des poids fait-on par epoch ?

- **A.** 390
- **B.** 128
- **C.** 391
- **D.** 50 000

Question 15 S4 · Evaluation

Quelles deux lignes le cours associe-t-il toujours pour évaluer ou prédire ?

- **A.** `model.train()` et `loss.backward()`
- **B.** `model.eval()` et `with torch.no_grad():`
- **C.** `optimizer.zero_grad()` et `optimizer.step()`
- **D.** `torch.manual_seed(0)` et `shuffle=True`

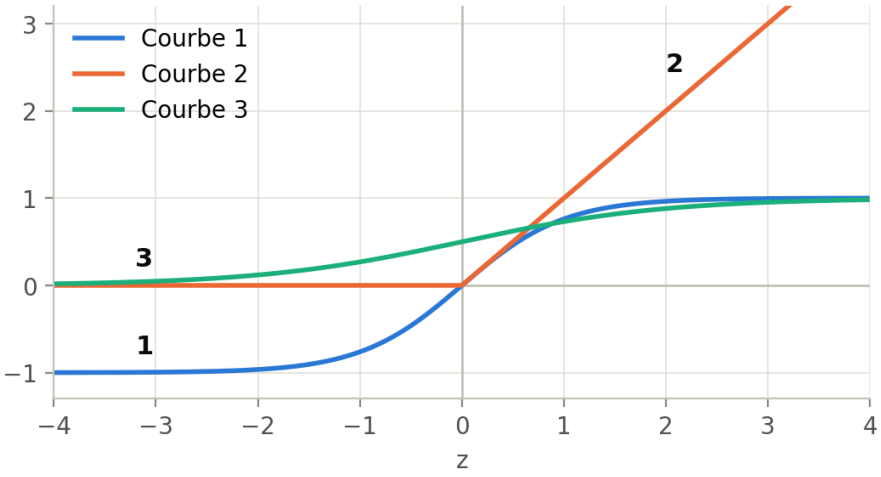
Question 16 S4 · nn.Linear

Quelle est la forme de `nn.Linear(784, 128).weight` ?

- **A.** (128, 784)
- **B.** (784, 128)
- **C.** (128,)
- **D.** (784,)

Question 17 S5 · Activations

Associe chaque courbe à sa fonction d'activation.



- **A.** 1 : sigmoïde · 2 : ReLU · 3 : tanh
- **B.** 1 : tanh · 2 : sigmoïde · 3 : ReLU
- **C.** 1 : tanh · 2 : ReLU · 3 : sigmoïde
- **D.** 1 : ReLU · 2 : tanh · 3 : sigmoïde

Question 18 S5 · ReLU

Que vaut la dérivée de ReLU(z) = max(0, z) ?

- **A.** z si z > 0, et 0 si z < 0
- **B.** σ(z)(1 - σ(z)) pour tout z
- **C.** 1 si z > 0, et 0 si z < 0
- **D.** 1/4 au maximum, atteint en z = 0

Question 19 S5 · Sigmoid derivative

Quel est le maximum de σ'(z) = σ(z)(1 - σ(z)), et où est-il atteint ?

- **A.** 1/4, atteint en z = 0
- **B.** 1, atteint en z = 0
- **C.** 1/2, atteint en z = 1
- **D.** 1/4, atteint quand z → +∞

Question 20 S5 · Vanishing gradient

Avec 6 couches cachées sigmoïdes au lieu d'une, l'accuracy de validation reste à 10 % et la loss à 2.30. Quelle est la cause ?

- **A.** Un bug dans la boucle d'entraînement
- **B.** Un réseau qui manque de paramètres pour apprendre
- **C.** Un vanishing gradient vers les premières couches
- **D.** Un learning rate beaucoup trop grand pour ce réseau

Question 21 S6 · Diagnosis

Pendant l'entraînement, la train loss continue de baisser alors que la validation loss remonte. Comment le cours appelle-t-il ce comportement ?

- **A.** De l'underfitting (high bias)
- **B.** De l'overfitting (high variance)
- **C.** Un learning rate beaucoup trop petit
- **D.** Un vanishing gradient

Question 22 S6 · Diagnosis

La loss devient énorme, puis `nan`, dès la première epoch. Selon le tableau de diagnostic du cours, quelle est la première chose à changer ?

- **A.** Multiplier le learning rate η par 10
- **B.** Diviser le learning rate η par 10
- **C.** Ajouter des couches cachées
- **D.** Entraîner pendant plus d'epochs

Question 23 S6 · Weight decay

Le weight decay (régularisation L2) remplace la loss J(W) par :

- **A.** $J(W) + (\lambda/2)\ \|W\|^2$
- **B.** $J(W) - (\lambda/2)\ \|W\|^2$
- **C.** $J(W) \times \lambda\ \|W\|^2$
- **D.** $J(W) + \lambda\ \|\nabla J\|^2$

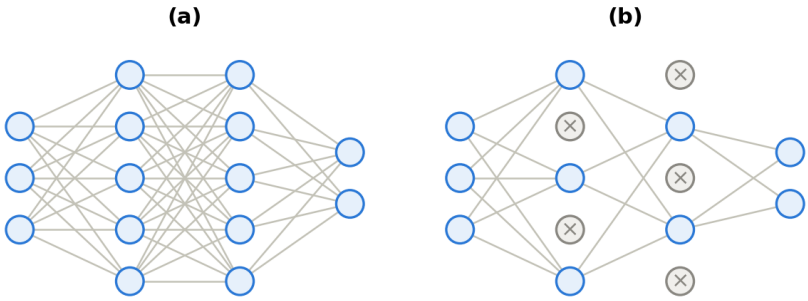
Question 24 S6 · Dropout

Dans `nn.Dropout(p=0.5)`, que représente `p` ?

- **A.** La probabilité de garder une valeur
- **B.** La proportion de couches désactivées
- **C.** La probabilité de mettre une valeur à 0
- **D.** Le facteur qui multiplie les poids gardés

Question 25 S6 · Dropout

Le schéma (b) montre le réseau (a) avec du dropout : certains neurones cachés sont mis à 0. Quand ce mécanisme est-il actif ?



- **A.** Seulement à l'entraînement, en mode `model.train()`
- **B.** Seulement à l'évaluation, en mode `model.eval()`
- **C.** À l'entraînement et à l'évaluation
- **D.** Seulement pendant la première epoch d'entraînement

Question 26 S6 · Early stopping

L'early stopping du cours arrête l'entraînement quand :

- **A.** la train loss stagne pendant `patience` epochs
- **B.** la train accuracy atteint 100 % sur tout le train set
- **C.** le learning rate descend sous 10^{-5}
- **D.** la validation loss stagne pendant `patience` epochs

Question 27 S6 · Batch norm

Batch norm : pour un neurone, sur un mini-batch, on calcule $\hat{z} = (z - \mu_B) / \sqrt{(\sigma_B^2 + \varepsilon)}$. Quelles statistiques a $\hat{z}$ sur le batch ?

- **A.** Moyenne 1 et variance 0
- **B.** Moyenne 0 et variance 1
- **C.** Valeurs toutes entre 0 et 1
- **D.** Moyenne 0 et variance σ_B^2

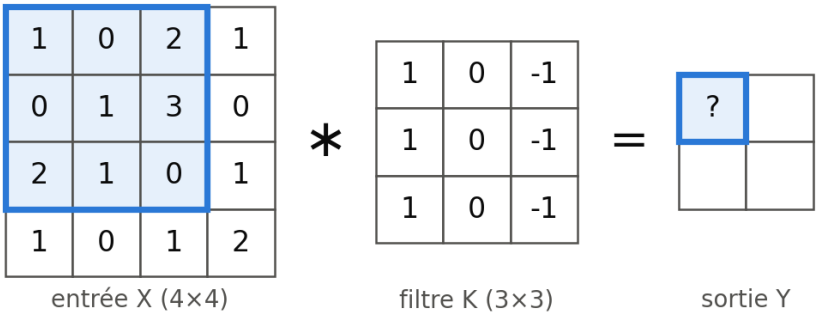
Question 28 S6 · Adam

Quels sont les hyperparamètres par défaut d'Adam donnés dans le cours ?

- **A.** $\eta = 0.1$, $\beta_1 = 0.5$, $\beta_2 = 0.9$
- **B.** $\eta = 1$, $\beta_1 = 0.99$, $\beta_2 = 0.5$
- **C.** $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
- **D.** $\eta = 10^{-5}$, $\beta_1 = 0.999$, $\beta_2 = 0.9$

Question 29 S7 · Convolution

On applique le filtre K à l'entrée X (stride 1, sans padding). Que vaut la case « ? » de la sortie, calculée sur la fenêtre bleue ?



- **A.** -2
- **B.** 2
- **C.** -5
- **D.** 3

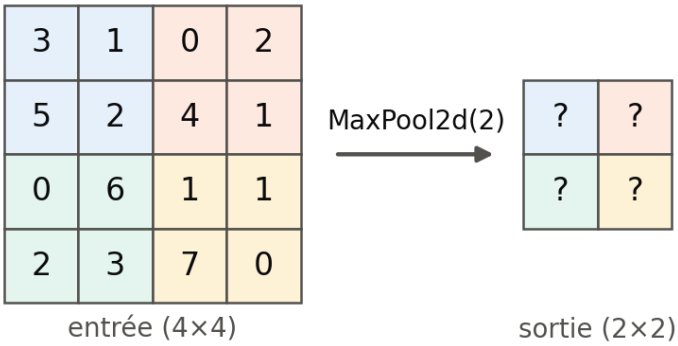
Question 30 S7 · Output size

Une image 28×28 passe dans `nn.Conv2d(1, 16, 3, padding=1)` (stride 1). Quelle est la taille spatiale de la sortie ?

- **A.** 26×26
- **B.** 28×28
- **C.** 30×30
- **D.** 14×14

Question 31 S7 · Pooling

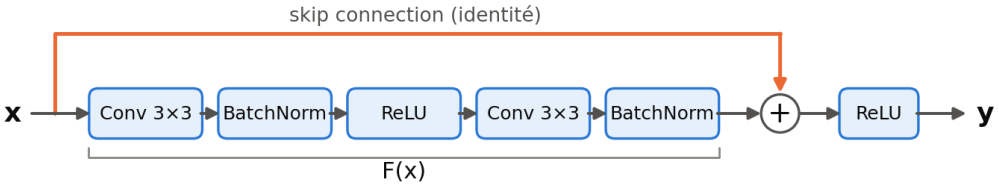
Que donne `nn.MaxPool2d(2)` (fenêtres 2×2, stride 2) sur cette entrée ?



- **A.** [[2.75, 1.75], [2.75, 2.25]]
- **B.** [[3, 0], [0, 1]]
- **C.** [[5, 4], [6, 7]]
- **D.** [[1, 0], [0, 0]]

Question 32 S7 · ResNet

Que calcule ce bloc résiduel de ResNet ?



- **A.** y = ReLU(F(x)) × x
- **B.** y = F(x) + ReLU(x)
- **C.** y = ReLU(F(x) + x)
- **D.** y = ReLU(F(x + x))

Question 33 S7 · Transfer learning

En transfer learning, comment « geler » le backbone pré-entraîné en PyTorch ?

- **A.** Mettre `p.requires_grad = False` pour ses paramètres
- **B.** Appeler `model.eval()` avant chaque epoch d'entraînement
- **C.** Mettre son learning rate à 1
- **D.** Supprimer ses couches de convolution

Question 34 S7 · Weight sharing

TP07 : on décale les images de test de 3 pixels. Le CNN perd beaucoup moins d'accuracy que le réseau dense. Pourquoi ?

- **A.** Il a beaucoup plus de paramètres que le réseau dense
- **B.** Le même filtre glisse sur toutes les positions
- **C.** Ses activations sont des sigmoïdes
- **D.** Il a vu des images décalées à l'entraînement

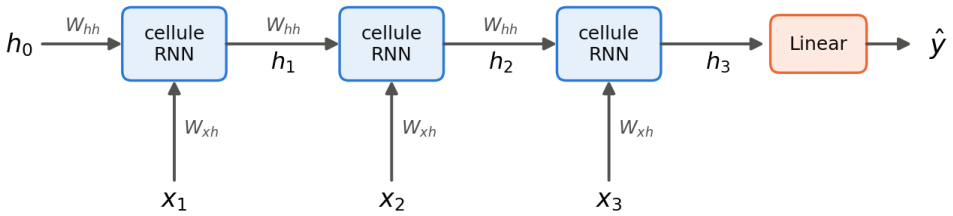
Question 35 S8 · RNN

Quelle est la récurrence d'un RNN simple vue en cours ?

- **A.** $h_t = \tanh(W_{xh}\, x_t + b)$
- **B.** $h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{t-1} + b)$
- **C.** $h_t = \sigma(W_{hh}\, h_{t+1} + W_{xh}\, x_{t+1} + b)$
- **D.** $h_t = \mathrm{ReLU}(W_{xh}\, x_t) \times W_{hh}\, h_{t-1}$

Question 36 S8 · RNN

Sur ce RNN déroulé sur 3 pas de temps, combien de matrices de poids différentes la récurrence utilise-t-elle (sans compter la couche Linear) ?



- **A.** 2 : W_{xh} et W_{hh}, partagées à chaque pas
- **B.** 6 : une paire de matrices par pas de temps
- **C.** 3 : une matrice par pas de temps
- **D.** 1 : la même pour x_t et pour h_{t-1}

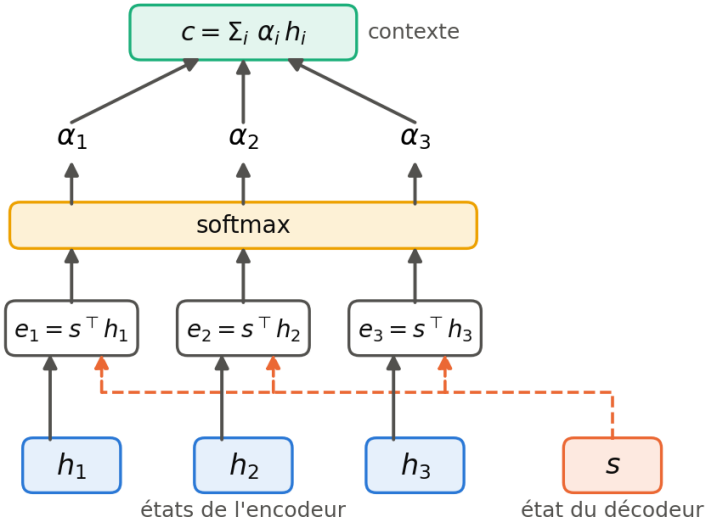
Question 37 S8 · LSTM

Dans une LSTM, à quoi sert la forget gate f_t ?

- **A.** À choisir la part de c_t montrée dans h_t
- **B.** À calculer le nouveau contenu candidat g_t
- **C.** À remettre h_t à 0 à chaque séquence
- **D.** À choisir la part de c_{t-1} que l'on garde

Question 38 S8 · Attention

Dans ce schéma d'attention, comment obtient-on les poids α_i ?



- **A.** Sigmoïde de chaque h_i séparément : ils valent 0 ou 1
- **B.** Moyenne des h_i : ils valent tous 1/3
- **C.** Softmax des scores $e_i = s \cdot h_i$: ils somment à 1
- **D.** Produit des h_i : ils somment à 0

Question 39 S8 · Sentence embeddings

Pour retrouver l'œuvre dont la description ressemble le plus à la question d'un visiteur, le TP08 compare les sentence embeddings avec :

- **A.** la distance de Hamming
- **B.** le nombre de mots communs
- **C.** la log loss
- **D.** la similarité cosinus

Question 40 S8 · Gradient clipping

Pourquoi le gradient clipping est-il standard pour les RNN ?

- **A.** Les RNN n'ont aucune fonction d'activation interne
- **B.** Le clipping remplace la backpropagation
- **C.** Il empêche le dropout de s'activer
- **D.** Le gradient à travers le temps peut exploser

Corrigé : QCM 1 : niveau facile

Compte 1 point par bonne réponse. Repère : 32/40 ou plus, les bases sont solides ; moins de 24/40, relis les sessions des questions ratées (voir les tags).

N°	Réponse	Explication
1	**D**	$\sigma(0) = 1/(1 + 1) = 0.5$; $\sigma \to 1$ quand $z \to +\infty$ et $\sigma \to 0$ quand $z \to -\infty$. On l'interprète comme une probabilité : $a = P(y = 1 \mid x)$. Symétrie utile : $\sigma(-z) = 1 - \sigma(z)$.
2	**B**	Un seul terme est actif : $y = 1$ donne $\mathcal{L} = -\log a$, $y = 0$ donne $\mathcal{L} = -\log(1 - a)$. Une prédiction fausse et confiante coûte très cher ($-\log a \to \infty$ quand $a \to 0$) ; une prédiction juste et confiante ne coûte presque rien.
3	**B**	$\partial\mathcal{L}/\partial a = (a - y)/(a(1 - a))$, $\partial a/\partial z = a(1 - a)$ et $\partial z/\partial w = x$: le facteur $a(1 - a)$ se simplifie. « The gradient is error times input: $(a - y)x$. » Et $\partial\mathcal{L}/\partial b = a - y$.
4	**D**	« The gradient points uphill, so we go in the opposite direction. » η est le learning rate : trop petit, c'est lent ; trop grand, on dépasse le minimum (overshoot). De même $b \leftarrow b - \eta\, \partial J/\partial b$.
5	**D**	$\sigma(z) \geq \sigma(0) \iff z \geq 0$: la frontière est encore la droite $w \cdot x + b = 0$, quel que soit l'entraînement. D'où la limite de la session 2 (« still one straight line ») : environ 0.885 sur les moons, et XOR reste impossible.
6	**A**	h_1 = OR et h_2 = NAND tracent deux droites ; XOR vaut 1 exactement entre les deux. Le neurone de sortie calcule AND(h_1, h_2) : il prend h en entrée, pas x.
7	**B**	$W^{[l]}$ a la forme $(n^{[l]}, n^{[l-1]})$: une ligne par neurone de la couche, une colonne par entrée (« sortie, entrée »), comme le weight de nn.Linear(3, 4). Ici $W^{[1]}$: (4, 3) et $W^{[2]}$: (2, 4).
8	**D**	$W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} = W'x + b'$: « A stack of linear layers is one linear layer ». Le cours parle d'un bug classique : le code tourne, mais le modèle est linéaire.
9	**A**	« forward: values flow to the right; backward: gradients flow to the left ». La backpropagation applique la chain rule en partant de la fin et réutilise chaque résultat pour l'étape suivante.
10	**A**	Chaque opération sur ce tenseur est enregistrée dans un graphe. `loss.backward()` parcourt ce graphe de la loss vers les paramètres (reverse mode) et range $\partial\mathcal{L}/\partial w$ dans `w.grad`.
11	**C**	Les 5 étapes : `optimizer.zero_grad()`, `logits = model(xb)`, `loss = loss_fn(logits, yb)`, `loss.backward()`, `optimizer.step()`. `zero_grad()` doit venir avant `backward()`, car PyTorch additionne les gradients.
12	**D**	nn.CrossEntropyLoss applique elle-même la softmax puis le -log : en ajouter une dans le modèle la ferait appliquer deux fois. Pas d'activation après la couche de sortie.
13	**D**	`.data` est un tenseur uint8 de forme (N, 28, 28), valeurs de 0 à 255. Les poids du réseau sont en float32 : on convertit et on ramène les pixels dans [0, 1]. Les labels restent en int64.
14	**C**	Une mise à jour par mini-batch : $\lceil 50\,000 / 128 \rceil = 391$ batchs (le dernier ne contient que $50\,000 - 390 \times 128 = 80$ images).
15	**B**	`model.eval()` met dropout et batch norm en mode évaluation ; `torch.no_grad()` évite d'enregistrer le graphe (plus rapide, moins de mémoire). Puis `pred = model(X).argmax(dim=1)`.
16	**A**	nn.Linear(n_in, n_out) calcule X @ W.T + b avec un weight de forme (n_out, n_in) = (128, 784), et un biais de forme (128,).
17	**C**	tanh va de -1 à 1 et vaut 0 en 0 (courbe 1). ReLU = max(0, z) : 0 pour $z < 0$, puis une droite (courbe 2). La sigmoïde va de 0 à 1 et vaut 0.5 en 0 (courbe 3).
18	**C**	Sur un chemin actif ($z > 0$), la dérivée vaut exactement 1 : le produit des dérivées ne rétrécit pas avec la profondeur. ReLU ne sature pas pour $z > 0$, coûte une simple comparaison et reste non linéaire (le coude en 0).
19	**A**	Avec $s = \sigma(z)$, $s(1 - s)$ est maximal pour $s = 1/2$, c'est-à-dire en $z = 0$: $\sigma'(0) = 1/4$. Même dans le meilleur cas, chaque couche sigmoïde divise le gradient par 4. Et $\sigma'(5) \approx 0.0066$ (saturation).
20	**C**	« More parameters, not a bug in the code. » Le gradient qui arrive à la première couche est un produit de facteurs $w\,\sigma'(z)$ avec $\sigma' \leq 1/4$: il disparaît (3.6×10^{-5} contre 0.77 pour la couche de sortie). C'est l'optimisation qui échoue, pas la capacité.

N°	Réponse	Explication
21	**B**	Train loss qui baisse et validation loss qui remonte : le réseau mémorise son train set et devient trop confiant sur les images de validation (overfitting, high variance). L'underfitting donne deux losses hautes et proches.
22	**B**	« nan means η too large » : on divise d'abord η par 10, puis on ajoute du gradient clipping et on vérifie que les entrées sont normalisées.
23	**A**	$J_{reg} = J + (\lambda/2)\,\|W\|^2$ pénalise les grands poids (les biais ne sont en général pas pénalisés). Le gradient gagne un terme λw : à chaque pas, le poids est d'abord multiplié par (1 - ηλ), il « décroît » vers 0.
24	**C**	p est la probabilité de DROP (mettre à 0) chaque valeur, indépendamment. La probabilité de garder est 1 - p, et les valeurs gardées sont multipliées par 1/(1 - p) (inverted dropout).
25	**A**	À chaque pas d'entraînement, un réseau « aminci » différent est entraîné. À l'évaluation (`model.eval()`), le dropout est l'identité : on utilise le réseau complet.
26	**D**	On surveille la validation loss : à chaque record on sauvegarde un checkpoint (`copy.deepcopy(model.state_dict())`), sinon `wait += 1`, et on s'arrête quand `wait >= patience`. On recharge ensuite le meilleur checkpoint.
27	**B**	$\hat{z}$ a une moyenne 0 et une variance 1 sur le batch ($\varepsilon \approx 10^{-5}$ évite la division par 0). Ensuite $y = \gamma\hat{z} + \beta$, avec γ et β appris (une paire par neurone) : le réseau peut choisir une autre échelle.
28	**C**	Adam (Kingma et Ba, 2015) : $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. C'est « momentum plus un pas par paramètre ».
29	**A**	On multiplie case par case la fenêtre 3×3 et K, puis on somme : colonne de gauche (1 + 0 + 2) × 1 = 3, colonne du milieu × 0 = 0, colonne de droite (2 + 3 + 0) × (-1) = -5. Total : 3 - 5 = -2.
30	**B**	$n_{out} = \lfloor (n + 2p - k)/s \rfloor + 1 = (28 + 2 - 3)/1 + 1 = 28$. Un padding de 1 avec un filtre 3×3 garde la taille (« same » padding).
31	**C**	On garde le maximum de chaque fenêtre 2×2 colorée : max(3, 1, 5, 2) = 5, max(0, 2, 4, 1) = 4, max(0, 6, 2, 3) = 6, max(1, 1, 7, 0) = 7. Les valeurs 2.75... seraient un average pooling.
32	**C**	La skip connection ajoute l'entrée x à la sortie de la branche F, puis on applique ReLU : y = ReLU(F(x) + x). Si le bloc est inutile, il lui suffit d'apprendre F(x) ≈ 0.
33	**A**	Avec `requires_grad = False`, autograd ne calcule plus le gradient de ces paramètres et l'optimiseur ne les touche pas. `model.eval()` change seulement le comportement du dropout et de la batch norm.
34	**B**	Weight sharing : la convolution applique le même filtre partout (translation equivariance). Un réseau dense a un poids par position de pixel : si l'objet bouge, ce sont d'autres poids qui travaillent. Le CNN a d'ailleurs environ 11 fois MOINS de paramètres.
35	**B**	Le nouvel état caché combine l'entrée courante x_t et l'état précédent h_{t-1}, puis passe dans tanh. Les mêmes W_{xh}, W_{hh} et b servent à chaque pas de temps.
36	**A**	Les poids sont partagés dans le temps (comme le weight sharing des CNN dans l'espace) : un RNN traite une séquence de n'importe quelle longueur avec un nombre fixe de paramètres.
37	**D**	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$: f_t (une sigmoïde, entre 0 et 1) dit quelle part de l'ancienne mémoire garder. C'est l'output gate o_t qui règle ce qui est montré dans h_t.
38	**C**	Score = produit scalaire entre l'état du décodeur s et chaque état h_i, puis softmax : $\alpha_i \geq 0$ et $\Sigma\,\alpha_i = 1$. Le contexte est la moyenne pondérée $c = \Sigma\,\alpha_i\,h_i$.
39	**D**	$\cos(u, v) = u \cdot v / (\|u\|\,\|v\|)$: 1 pour la même direction, 0 pour des vecteurs orthogonaux. On prend l'argmax de chaque ligne de la matrice des cosinus. Les embeddings captent le sens, pas seulement les mots communs (« ocean » et « wave »).
40	**D**	En backpropagation through time, le gradient est un produit de facteurs sur T pas : il peut exploser (ou disparaître). Le clipping limite sa norme à un seuil c en gardant sa direction.

QCM 2 : niveau moyen

Calculs du cours avec de nouvelles valeurs, formes de tenseurs, lecture de courbes et de code PyTorch.

Grille de réponses

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**

Question 1 S2 · Sigmoid neuron

Neurone sigmoïde avec w = (1, 2), b = -1 et x = (2, -1). Sachant que e ≈ 2.718, que vaut a, et quelle classe est prédite ?

- **A.** a ≈ 0.269 : classe 0
- **B.** a ≈ 0.731 : classe 1
- **C.** a ≈ 0.269 : classe 1
- **D.** a = -1 : classe 0

Question 2 S2 · Log loss

Deux exemples : (y = 1, a = 0.9) et (y = 0, a = 0.2). Avec ln 0.9 ≈ -0.105, ln 0.8 ≈ -0.223 et ln 0.2 ≈ -1.609, que vaut le coût J (la log loss moyenne) ?

- **A.** ≈ 0.328
- **B.** ≈ 0.857
- **C.** ≈ 0.164
- **D.** ≈ 0.025

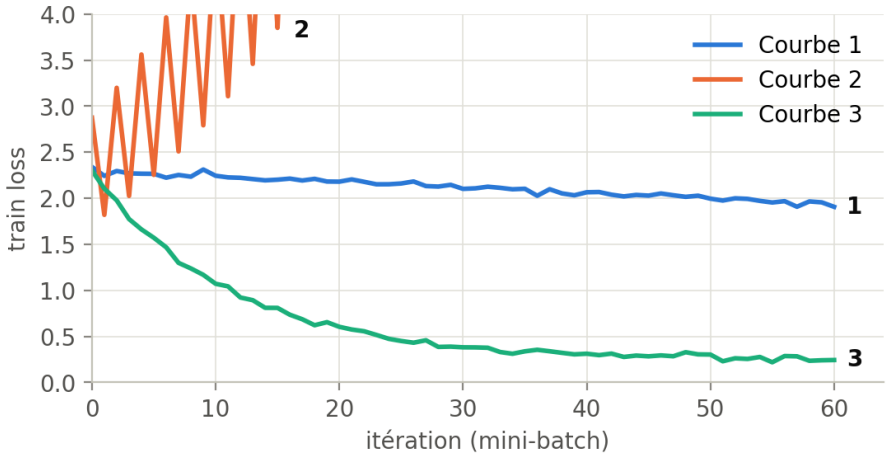
Question 3 S2 · Vectorisation

X a la forme (m, n) (un exemple par ligne), A et y la forme (m,). Quelle ligne NumPy calcule dw ?

- **A.** `dw = X.T @ (A - y) / m`
- **B.** `dw = X @ (A - y) / m`
- **C.** `dw = (A - y) @ X.T / m`
- **D.** `dw = np.mean(A - y)`

Question 4 S2 · Learning rate

Même modèle, mêmes données : seul le learning rate η change. Associe les courbes.



- **A.** 1 : η trop grand · 2 : η trop petit · 3 : η bien choisi
- **B.** 1 : η bien choisi · 2 : η trop grand · 3 : η trop petit
- **C.** 1 : η trop petit · 2 : η bien choisi · 3 : η trop grand
- **D.** 1 : η trop petit · 2 : η trop grand · 3 : η bien choisi

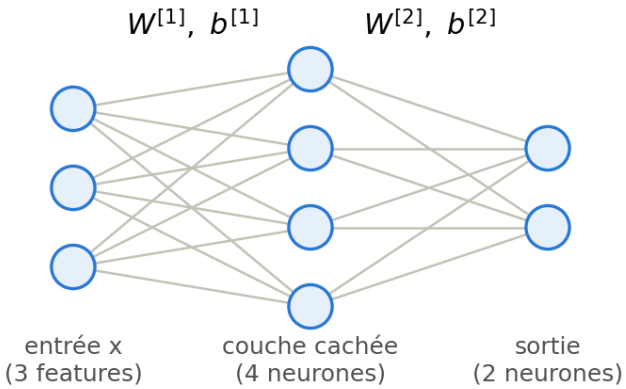
Question 5 S2 · Logits

Pourquoi le modèle sort-il le logit z avec `nn.BCEWithLogitsLoss`, plutôt que `torch.sigmoid(z)` suivi d'un -log ?

- **A.** Parce que les logits sont toujours entre 0 et 1
- **B.** Pour annuler le gradient des exemples faciles
- **C.** En float32, σ(-200) vaut 0 et log 0 = -∞
- **D.** Parce que la sigmoïde n'est pas dérivable en 0

Question 6 S3 · Parameters

Combien de paramètres (poids et biais) a ce réseau 3-4-2 ?



- **A.** 20
- **B.** 26
- **C.** 24
- **D.** 18

Question 7 S3 · Forward pass

Réseau XOR à activation marche (1 si $z \geq 0$) : h_1 : w = (1, 1), b = -0.5 ; h_2 : w = (-1, -1), b = 1.5 ; sortie : w = (1, 1), b = -1.5, appliquée à h. Pour x = (1, 1), que valent h et $\hat{y}$?

- **A.** h = (1, 1) et $\hat{y} = 1$
- **B.** h = (1, 0) et $\hat{y} = 0$
- **C.** h = (0, 1) et $\hat{y} = 0$
- **D.** h = (0, 0) et $\hat{y} = 1$

Question 8 S3 · Backpropagation

Pour la couche cachée sigmoïde d'un réseau à 2 couches, quelle formule donne $dz^{[1]}$?

- **A.** $W^{[1]T}\, dz^{[2]} \odot a^{[1]} \odot (1 - a^{[1]})$
- **B.** $dz^{[2]}\, a^{[1]T}$
- **C.** $W^{[2]T}\, dz^{[2]} \odot a^{[1]} \odot (1 - a^{[1]})$
- **D.** $W^{[2]}\, dz^{[2]} + a^{[1]}(1 - a^{[1]})$

Question 9 S3 · Autograd

Dans le cours, on écrit `loss = -(y*torch.log(a2) + (1 - y)*torch.log(1 - a2)).sum()` avant `loss.backward()`. Pourquoi ce `.sum()` ?

- **A.** `backward()` part d'un seul nombre ; a2 a la forme (1,)
- **B.** Pour additionner les gradients de toutes les couches
- **C.** Pour remettre les gradients à zéro avant le calcul
- **D.** Pour convertir la loss du float32 au float64

Question 10 S3 · Shapes

Dans un réseau 784-128-10, quelle forme a le gradient $dW^{[1]}$?

- **A.** (784, 128)
- **B.** (128, 1)
- **C.** (128, 784)
- **D.** (10, 128)

Question 11 S4 · Parameters

Combien de paramètres a `nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.Sigmoid(), nn.Linear(128, 10))` ?

- **A.** 101 632
- **B.** 100 480
- **C.** 101 770
- **D.** 1 290

Question 12 S4 · Shapes

Quelles sont les formes de `logits` et de `pred` ?

```
model.eval()
with torch.no_grad():
    logits = model(X_test)          # X_test : (10000, 28, 28)
    pred = logits.argmax(dim=1)
```

- **A.** (10000, 10) et (10,)
- **B.** (10000, 10) et (10000,)
- **C.** (10, 10000) et (10000,)
- **D.** (10000, 784) et (10000, 10)

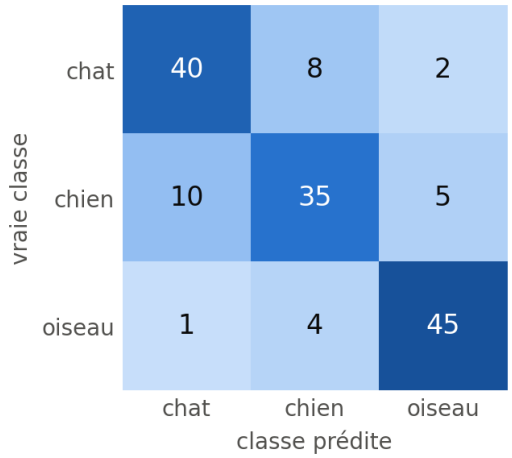
Question 13 S4 · Standardisation

On standardise les pixels avec x ← (x - μ)/s. Sur quelles images calcule-t-on μ et s ?

- **A.** Sur les images de validation et de test seulement
- **B.** Sur train, validation et test réunis
- **C.** Sur les images d'entraînement seulement
- **D.** Sur chaque mini-batch, séparément

Question 14 S4 · Evaluation

Matrice de confusion d'un classifieur (lignes : vraie classe, colonnes : classe prédite). Quelle est l'accuracy, et quelle est l'erreur la plus fréquente ?



- **A.** 0.80 ; « chat » prédit « chien » (8 fois)
- **B.** 0.85 ; « chien » prédit « oiseau » (5 fois)
- **C.** 0.27 ; « oiseau » prédit « chat » (1 fois)
- **D.** 0.80 ; « chien » prédit « chat » (10 fois)

Question 15 S4 · Training loop

Dans `train_one_epoch`, on accumule `running += loss.item() * len(yb)` puis on renvoie `running / total`. Pourquoi multiplier par `len(yb)` ?

- **A.** Pour convertir la loss moyenne du batch en accuracy
- **B.** Pour une vraie moyenne, même avec un petit dernier batch
- **C.** Pour compenser l'effet du learning rate sur la loss du batch
- **D.** Pour que `backward()` reçoive bien un seul nombre

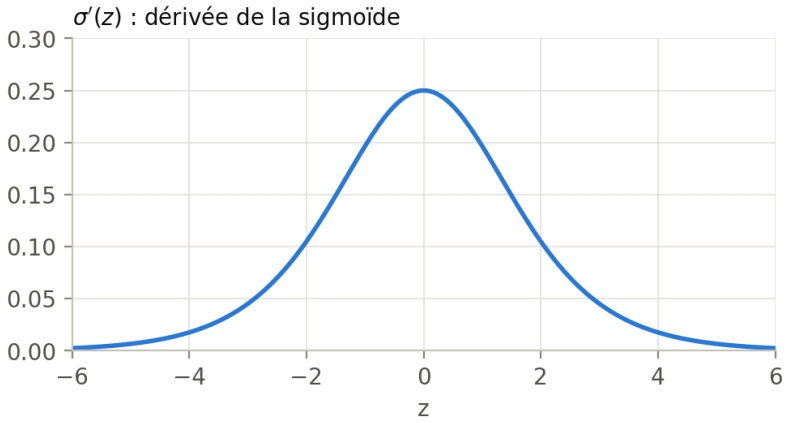
Question 16 S4 · Data splits

Pourquoi garder un jeu de validation distinct du jeu de test ?

- **A.** La validation choisit les réglages ; le test sert à la fin
- **B.** La validation sert à calculer les gradients des poids
- **C.** Le test sert à choisir le learning rate et l'architecture du réseau
- **D.** C'est le même jeu de données, présenté sous deux noms différents

Question 17 S5 · Vanishing gradient

D'après cette courbe, dans une chaîne de 5 couches sigmoïdes avec tous les poids égaux à 1, le gradient qui arrive à la première couche est multiplié, au mieux, par :



- **A.** $(1/2)^5 \approx 3 \times 10^{-2}$
- **B.** $5 \times 1/4 \approx 1.25$
- **C.** $1^5 = 1$
- **D.** $(1/4)^5 \approx 10^{-3}$

Question 18 S5 · Dead ReLU

Un neurone ReLU a $z = w \cdot x + b < 0$ pour TOUTES les entrées du dataset. Que se passe-t-il ?

- **A.** Il sort -z à la place de z et apprend donc à l'envers
- **B.** Son gradient vaut 1 partout : il finit par se réparer tout seul
- **C.** Il sort 0.01 z grâce à la pente de la ReLU
- **D.** Il sort 0 et son gradient est nul : il ne revient jamais

Question 19 S5 · Which activation where

Laquelle de ces associations correspond au tableau « Which activation where? » du cours ?

- **A.** Sigmoïde : couches cachées d'un réseau profond
- **B.** Softmax : sortie pour C classes exclusives
- **C.** ReLU : sortie d'un classifieur à 2 classes
- **D.** tanh : sortie d'un classifieur à 10 classes

Question 20 S5 · Backpropagation

$z_1 = w_1x + b_1$, $a_1 = \text{ReLU}(z_1)$, $z_2 = w_2a_1 + b_2$, $a_2 = \text{ReLU}(z_2)$, $\hat{y} = w_3a_2$, $\mathcal{L} = \frac{1}{2}(\hat{y} - y)^2$, avec $x = 2$, $y = 3$, $w_1 = 0.5$, $b_1 = 1$, $w_2 = -1$, $b_2 = 3$, $w_3 = 2$. Que vaut $\partial\mathcal{L}/\partial w_3$?

- **A.** -1
- **B.** 1
- **C.** -2
- **D.** 0.5

Question 21 S6 · Weight decay

SGD avec weight decay : $w = (4, -2)$, $\partial J/\partial w = (1, 0)$, $\eta = 0.1$ et $\lambda = 0.5$. Que vaut w après un pas ?

- **A.** (3.9, -2.0)
- **B.** (3.8, -1.95)
- **C.** (1.9, -1.0)
- **D.** (3.7, -1.9)

Question 22 S6 · Dropout

Inverted dropout avec p = 0.2 (probabilité de drop). Activations a = (4, 2, 8, 1) et masque tiré m = (1, 0, 1, 1). Que transmet la couche en mode train ?

- **A.** (3.2, 0, 6.4, 0.8)
- **B.** (5, 0, 10, 1.25)
- **C.** (4, 0, 8, 1)
- **D.** (20, 0, 40, 5)

Question 23 S6 · Batch norm

Batch norm sur un neurone, mini-batch z = (2, 2, 6, 6), γ = 3, β = 1, ε négligé. Que vaut la sortie y = γẑ + β ?

- **A.** (-2, -2, 4, 4)
- **B.** (-1, -1, 1, 1)
- **C.** (-3, -3, 3, 3)
- **D.** (-0.5, -0.5, 2.5, 2.5)

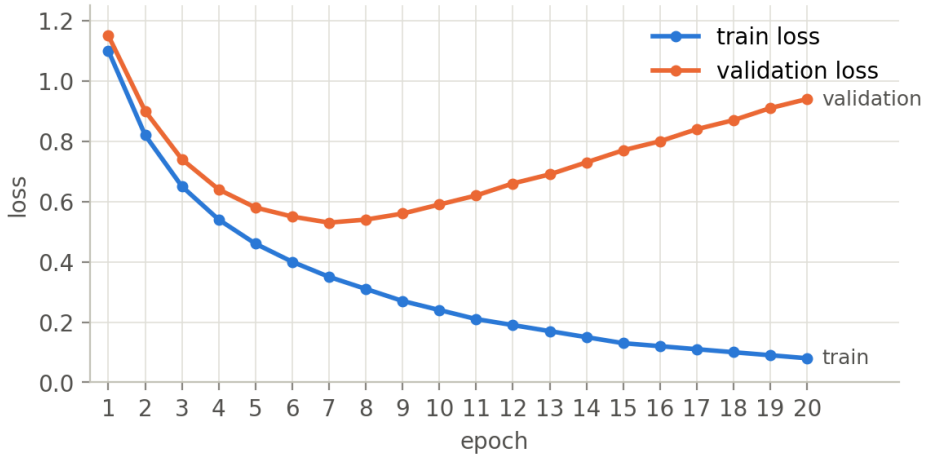
Question 24 S6 · Batch norm

En mode `model.eval()`, une couche `nn.BatchNorm1d` normalise avec :

- **A.** les running mean et running var de l'entraînement
- **B.** la moyenne et la variance du batch d'évaluation
- **C.** une moyenne 0 et une variance 1 fixées d'avance
- **D.** seulement γ et β, sans aucune normalisation

Question 25 S6 · Early stopping

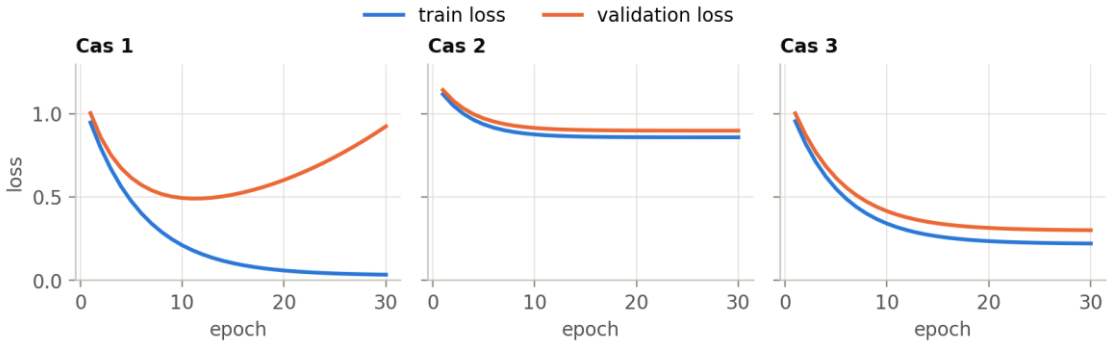
Early stopping avec `patience = 3` sur ces courbes. Quand l'entraînement s'arrête-t-il, et quel modèle garde-t-on ?



- **A.** Arrêt après l'epoch 10, on recharge l'epoch 7
- **B.** Arrêt après l'epoch 7, on garde l'epoch 7
- **C.** Arrêt après l'epoch 10, on garde l'epoch 10
- **D.** Arrêt après l'epoch 20 : on garde le dernier modèle

Question 26 S6 · Diagnosis

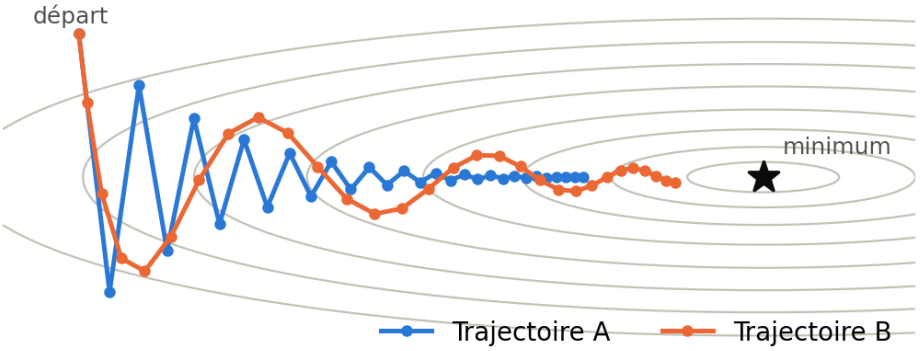
Associe chaque cas à son diagnostic.



- **A.** 1 : underfitting · 2 : overfitting · 3 : bon réglage
- **B.** 1 : overfitting · 2 : bon réglage · 3 : underfitting
- **C.** 1 : overfitting · 2 : underfitting · 3 : bon réglage
- **D.** 1 : bon réglage · 2 : underfitting · 3 : overfitting

Question 27 S6 · Momentum

La trajectoire A est une SGD simple, la B utilise du momentum. D'après le cours, pourquoi B progresse-t-elle plus vite vers le minimum ?



- **A.** Le learning rate diminue tout seul à chaque pas
- **B.** Les composantes qui changent de signe s'annulent dans v
- **C.** Le gradient est calculé sur tout le dataset
- **D.** Les poids sont d'abord multipliés par (1 - ηλ) à chaque pas

Question 28 S6 · Diagnosis

Sur FashionMNIST (10 classes), la loss reste plate à 2.30 dès le début. Que signifie cette valeur ?

- **A.** ln 10 : le réseau prédit 1/10 pour chaque classe
- **B.** La plus petite loss possible avec 10 classes
- **C.** Le signe d'un learning rate beaucoup trop grand
- **D.** La loss d'un réseau qui a mémorisé le train set

Question 29 S7 · Parameters

Combien de paramètres appris a la couche `nn.Conv2d(3, 16, 5)` ?

- **A.** 1 200
- **B.** 416
- **C.** Ça dépend de la taille de l'image
- **D.** 1 216

Question 30 S7 · Shapes

Quelle forme ce code affiche-t-il ?

```
x = torch.randn(1, 3, 32, 32)      # (N, C, H, W)
conv = nn.Conv2d(3, 8, 5)
pool = nn.MaxPool2d(2)
print(pool(conv(x)).shape)
```

- **A.** torch.Size([1, 8, 16, 16])
- **B.** torch.Size([1, 3, 14, 14])
- **C.** torch.Size([1, 8, 28, 28])
- **D.** torch.Size([1, 8, 14, 14])

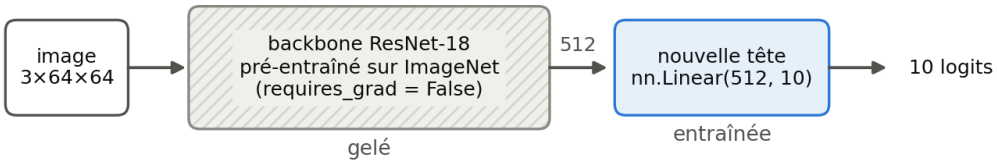
Question 31 S7 · Output size

Taille de sortie d'une convolution sur une image 100×100 avec k = 3, p = 1 et s = 2 ?

- **A.** 49×49
- **B.** 51×51
- **C.** 50×50
- **D.** 98×98

Question 32 S7 · Transfer learning

Le backbone est gelé et seule la nouvelle tête est entraînée. Combien de paramètres l'optimiseur met-il à jour ?



- **A.** 5 120
- **B.** 11 181 642
- **C.** 522
- **D.** 5 130

Question 33 S7 · Transfer learning

Pour fine-tuner le dernier bloc, le cours crée `Adam([{"params": model.layer4.parameters(), "lr": 1e-4}, {"params": model.fc.parameters(), "lr": 1e-3}])`. Pourquoi un pas plus petit pour `layer4` ?

- **A.** Parce que `layer4` contient moins de paramètres
- **B.** Pour que `layer4` reste presque gelée pendant tout l'entraînement
- **C.** Pour ajuster ses poids pré-entraînés sans les détruire
- **D.** Parce qu'Adam refuse deux groupes au même lr

Question 34 S7 · ResNet

He et al. (2015) : sur CIFAR-10, un CNN « plain » de 56 couches a une erreur d'ENTRAÎNEMENT plus haute qu'un CNN de 20 couches. Comment le cours interprète-t-il ce résultat ?

- **A.** De l'overfitting dû au trop grand nombre de couches
- **B.** Un problème d'optimisation (degradation problem)
- **C.** Un manque de données d'entraînement
- **D.** Un bug : un réseau plus profond est toujours meilleur

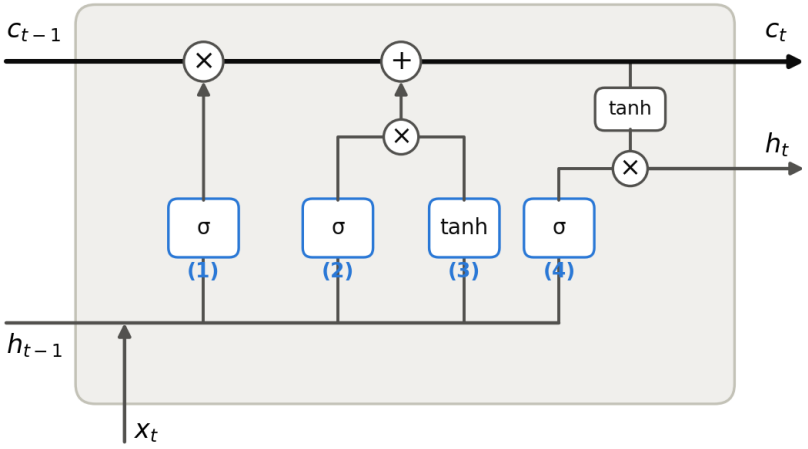
Question 35 S8 · RNN

RNN scalaire : $h_t = \tanh(w_x x_t + w_h h_{t-1})$, avec $w_x = 2$, $w_h = 1$, $h_0 = 0$ et $x = (0.5, -0.5)$. On a $h_1 = \tanh(1) \approx 0.762$. Que vaut h_2 ?

- **A.** ≈ -0.762
- **B.** ≈ -0.234
- **C.** ≈ -0.238
- **D.** ≈ 0.441

Question 36 S8 · LSTM

Sur ce schéma de cellule LSTM, que sont les blocs (1) et (4) ?



- **A.** (1) input gate i_t ; (4) forget gate f_t
- **B.** (1) output gate o_t ; (4) input gate i_t
- **C.** (1) candidat g_t ; (4) forget gate f_t
- **D.** (1) forget gate f_t ; (4) output gate o_t

Question 37 S8 · LSTM

Avec l'initialisation par défaut de PyTorch, la forget gate vaut $f \approx \sigma(0) = 0.5$. Quelle part du gradient du cell state survit-elle sur 30 pas ?

- **A.** Environ 0.5
- **B.** Environ 10^{-9}
- **C.** Environ 10^{-2}
- **D.** Environ 15

Question 38 S8 · LSTM

Combien de paramètres a `nn.LSTM(8, 32)` (8 entrées, 32 unités, deux vecteurs de biais par bloc comme dans PyTorch) ?

- **A.** 5 376
- **B.** 1 344
- **C.** 4 032
- **D.** 5 248

Question 39 S8 · Attention

Attention : $s = (2, 1)$, $h_1 = (1, 0)$, $h_2 = (0, 1)$, $h_3 = (1, 1)$. Quel état reçoit le plus grand poids α ?

- **A.** h_1, avec $\alpha \approx 0.67$
- **B.** h_2, avec $\alpha \approx 0.67$
- **C.** h_3, avec $\alpha \approx 0.67$
- **D.** Aucun : $\alpha = 1/3$ pour chacun

Question 40 S8 · Production checklist

Le GPU ne tient que des batchs de 16 images. Le code du cours divise la loss par `accum = 4` et ne fait `optimizer.step()` que tous les 4 batchs. Qu'obtient-on ?

- **A.** Un gradient 4 fois plus grand qu'un batch de 64
- **B.** Un learning rate automatiquement divisé par 4
- **C.** Quatre mises à jour indépendantes par batch
- **D.** Le même gradient qu'un seul batch de 64 images

Corrigé : QCM 2 : niveau moyen

Compte 1 point par bonne réponse. Repère : 30/40 ou plus, très bon niveau ; moins de 22/40, refais les exercices à la main des sessions concernées.

N°	Réponse	Explication
1	**A**	$z = 1 \times 2 + 2 \times (-1) - 1 = -1$, donc $a = \sigma(-1) = 1/(1 + e) \approx 0.269 < 0.5$: classe 0, avec $P(y = 1 \mid x) \approx 27\ \%$. Piège classique : $\sigma(-1)$ utilise e^{+1}. Et $\sigma(1) = 1 - \sigma(-1) \approx 0.731$.
2	**C**	$\mathcal{L}_1 = -\ln 0.9 \approx 0.105$ et $\mathcal{L}_2 = -\ln(1 - 0.2) = -\ln 0.8 \approx 0.223$. J est la MOYENNE : $(0.105 + 0.223)/2 \approx 0.164$. 0.328 est la somme ; 0.857 utilise -ln a pour l'exemple de classe 0 ; 0.025 est une MSE.
3	**A**	$dw = (1/m)\, X^T(A - y)$: X.T (n, m) @ (A - y) (m,) donne un vecteur (n,), de la forme de w. `np.mean(A - y)` est db. Les deux autres lignes n'ont pas des formes compatibles.
4	**D**	η trop petit : la loss bouge à peine (courbe 1). η trop grand : elle saute et explose (courbe 2, elle finira en `nan`). Un bon η est « large enough to be fast and small enough to be stable » (courbe 3).
5	**C**	Avec z = -200 et y = 1 : la sigmoïde donne 0, le -log donne `inf`, alors que BCEWithLogitsLoss donne 200. « The model outputs the logit z, and the loss applies the sigmoid itself », sous une forme stable.
6	**B**	Couche l : $n^{[l]}(n^{[l-1]} + 1)$ paramètres. Couche 1 : $4 \times (3 + 1) = 16$. Couche 2 : $2 \times (4 + 1) = 10$. Total 26 (20 si on oublie les biais).
7	**B**	$z_1 = 1 + 1 - 0.5 = 1.5$ donc $h_1 = 1$; $z_2 = -1 - 1 + 1.5 = -0.5$ donc $h_2 = 0$. Sortie : $z = 1 + 0 - 1.5 = -0.5$, donc $\hat{y} = 0 = \text{XOR}(1, 1)$. Piège : le neurone de sortie prend h en entrée, pas x.
8	**C**	$da^{[1]} = W^{[2]T}\, dz^{[2]}$ renvoie l'erreur de la sortie vers chaque neurone caché, pondérée par la connexion utilisée à l'aller (donc $W^{[2]}$, pas $W^{[1]}$) ; puis on multiplie terme à terme par $\sigma'(z^{[1]}) = a^{[1]}(1 - a^{[1]})$. $dz^{[2]}\, a^{[1]T}$ est $dW^{[2]}$.
9	**A**	`backward()` part d'un scalaire : a2 a la forme (1,), le `.sum()` en fait un seul nombre sans changer sa valeur. Autograd retrouve alors exactement les gradients calculés à la main (W2.grad = [[-0.25, -0.25]]).
10	**C**	« Each gradient has the shape of its parameter » : $W^{[1]}$ est (128, 784), donc $dW^{[1]}$ aussi. C'est une bonne vérification quand on code la backpropagation.
11	**C**	$784 \times 128 + 128 = 100\,480$ pour la première couche, $128 \times 10 + 10 = 1\,290$ pour la seconde : total 101 770. Flatten et Sigmoid n'ont aucun paramètre. 101 632 oublie les biais.
12	**B**	Le modèle renvoie 10 logits par image : (10000, 10). `argmax(dim=1)` prend l'indice du plus grand logit sur chaque ligne : une classe par image, forme (10000,).
13	**C**	Les deux nombres viennent des images d'entraînement uniquement, sinon il y a data leakage (le test influence le modèle). On applique ensuite la même transformation à la validation et au test.
14	**D**	La diagonale contient les bonnes prédictions : $(40 + 35 + 45)/150 = 120/150 = 0.80$. Hors diagonale, la plus grande case est ligne « chien », colonne « chat » : 10 chiens prédits chats. Dans le TP05, c'était Pullover prédit Coat (194 fois).
15	**B**	La loss d'un batch est déjà une moyenne sur ce batch. En la multipliant par la taille du batch puis en divisant par le total, on obtient la vraie moyenne sur l'epoch, même si le dernier batch ne contient que 80 images.
16	**A**	On compare les réglages sur la validation. Le test n'est mesuré qu'une fois, à la fin, sinon « le test deviendrait une deuxième validation » et son score serait trop optimiste.
17	**D**	Le gradient est un produit d'un facteur w $\sigma'(z)$ par couche, avec $\sigma'(z) \leq 1/4$ (maximum en z = 0). Au mieux $(1/4)^5 = 1/1024 \approx 10^{-3}$; avec 10 couches, $\approx 10^{-6}$.
18	**D**	Sortie 0 et dérivée 0 partout : aucun gradient n'atteint w et b, c'est une dead ReLU. Causes : learning rate trop grand, mauvaise initialisation. Remède possible : Leaky ReLU, max(0.01z, z), dont le gradient n'est jamais exactement nul.
19	**B**	ReLU : couches cachées (défaut). tanh : dans les cellules RNN et LSTM. Sigmoïde : sortie à 2 classes ou multi-label, et gates de LSTM (à éviter dans les couches cachées profondes). Softmax : sortie pour C classes exclusives (dans CrossEntropyLoss en PyTorch).

N°	Réponse	Explication
20	**A**	Forward : $z_1 = 2$, $a_1 = 2$, $z_2 = 1$, $a_2 = 1$, $\hat{y} = 2$, $\mathcal{L} = 0.5$. Puis $\partial\mathcal{L}/\partial\hat{y} = \hat{y} - y = -1$ et $\partial\mathcal{L}/\partial w_3 = (\hat{y} - y) \times a_2 = -1 \times 1 = -1$: « gradient qui arrive × entrée du poids ».
21	**D**	$w \leftarrow (1 - \eta\lambda)\, w - \eta\, \partial J/\partial w$ avec $1 - \eta\lambda = 0.95$: $(0.95 \times 4 - 0.1 \times 1,\ 0.95 \times (-2) - 0) = (3.7, -1.9)$. (3.9, -2.0) est le pas sans régularisation ; (1.9, -1.0) oublie le η devant λw.
22	**B**	En train : $\tilde{a} = m \times a / (1 - p) = m \times a / 0.8$, soit (5, 0, 10, 1.25). Diviser par 1 - p garde la même espérance ($E[\tilde{a}] = a$), donc il n'y a rien à changer en évaluation, où la couche renvoie a.
23	**A**	$\mu_B = 4$ et $\sigma_B^2 = (4 + 4 + 4 + 4)/4 = 4$, donc $\hat{z} = (z - 4)/2 = (-1, -1, 1, 1)$ et $y = 3\hat{z} + 1 = (-2, -2, 4, 4)$. La sortie a pour moyenne $\beta = 1$ et pour écart-type $\gamma = 3$. (-0.5, ...) vient d'une division par la variance au lieu de l'écart-type.
24	**A**	À l'entraînement, PyTorch tient une moyenne glissante des statistiques de batch (momentum = 0.1). En évaluation, on peut prédire une image seule (pas de batch) : on utilise ces running statistics. Encore une raison de ne jamais oublier `model.eval()`.
25	**A**	Le minimum de la validation loss est à l'epoch 7 (0.53). Les epochs 8, 9 et 10 ne battent pas ce record : `wait` atteint 3 = patience, on s'arrête, puis `model.load_state_dict(best_state)` revient à l'epoch 7.
26	**C**	Cas 1 : train loss vers 0 et validation qui remonte (overfitting : early stopping, weight decay, dropout, plus de données). Cas 2 : deux losses hautes et proches (underfitting : modèle plus gros, entraîner plus longtemps, η plus grand). Cas 3 : deux losses basses, petit écart.
27	**B**	$v \leftarrow \beta v + g$ puis $w \leftarrow w - \eta v$: v est une somme des gradients passés. Le zigzag (signe alterné) s'annule, la direction constante s'additionne : avec un gradient constant, v tend vers $g/(1 - \beta) = 10g$ pour $\beta = 0.9$.
28	**A**	Un réseau qui ne sait rien prédit 1/C partout : loss = $-\ln(1/C) = \ln C = \ln 10 = 2.30$. Causes possibles : η trop petit, bug dans la boucle (pas de `step()`, pas de `zero_grad()`), ReLU mortes, réseau sigmoïde profond. Premier test : overfitter un petit batch.
29	**D**	#params = $C_{out} \times (C_{in} \times k \times k + 1) = 16 \times (3 \times 25 + 1) = 16 \times 76 = 1\,216$. Chaque filtre couvre les 3 canaux d'entrée et a 1 biais. Ce nombre ne dépend pas de la taille de l'image (weight sharing).
30	**D**	Convolution sans padding : 32 - 5 + 1 = 28, avec 8 canaux de sortie (un par filtre). Le max pooling 2×2 divise la hauteur et la largeur par 2 et garde le nombre de canaux : (1, 8, 14, 14).
31	**C**	$n_{out} = \lfloor (n + 2p - k)/s \rfloor + 1 = \lfloor (100 + 2 - 3)/2 \rfloor + 1 = \lfloor 49.5 \rfloor + 1 = 50$. Attention à l'arrondi vers le bas (floor).
32	**D**	nn.Linear(512, 10) : 512 × 10 poids + 10 biais = 5 130. C'est le nombre vérifié dans le TP07 avec `count_parameters(model_tl, trainable_only=True)` ; le ResNet-18 complet en a environ 11 millions.
33	**C**	Parameter groups : un learning rate par groupe. Les poids de layer4 viennent d'ImageNet et sont déjà bons, on veut seulement les ajuster. La tête est neuve (poids aléatoires) et peut apprendre plus vite.
34	**B**	Ce n'est pas de l'overfitting (l'erreur d'entraînement elle-même est plus haute) : c'est un problème d'optimisation. Le réseau profond pourrait copier le petit avec des couches identité, mais la descente de gradient ne le trouve pas. ResNet fait de l'identité le comportement par défaut : $y = F(x) + x$.
35	**B**	$h_2 = \tanh(2 \times (-0.5) + 1 \times 0.762) = \tanh(-0.238) \approx -0.234$. Oublier h_1 donne $\tanh(-1) \approx -0.762$; oublier le tanh donne -0.238 ; 0.441 serait une sigmoïde.
36	**D**	(1) multiplie l'ancienne mémoire c_{t-1} : c'est f_t. (2) et (3) sont i_t (σ) et le candidat g_t (tanh), qui ajoutent du contenu : $c_t = f_t \odot c_{t-1} + i_t \odot g_t$. (4) multiplie $\tanh(c_t)$ pour donner $h_t = o_t \odot \tanh(c_t)$.
37	**B**	Le long du cell state, on multiplie par f à chaque pas : $0.5^{30} \approx 9 \times 10^{-10}$. Avec un biais de forget gate à 2, $f \approx \sigma(2) = 0.88$ et $0.88^{30} \approx 0.02$: environ 2 % survivent, soit environ 2×10^7 fois plus.
38	**A**	Un bloc = 32 × 8 + 32 × 32 + 2 × 32 = 1 344. Un RNN simple a 1 bloc, une GRU 3 (4 032), une LSTM 4 (gates i, f, g, o) : 4 × 1 344 = 5 376. 5 248 oublie le second vecteur de biais.

N°	Réponse	Explication
39	**C**	Scores $e_i = s \cdot h_i = (2, 1, 3)$. Softmax : $\alpha \approx (0.245, 0.090, 0.665)$. Le décodeur regarde surtout h_3, et le contexte vaut $c = \Sigma\, \alpha_i\, h_i \approx (0.910, 0.755)$.
40	**D**	Gradient accumulation : les gradients de 4 batchs s'ADDITIONNENT (pas de `zero_grad()` entre eux), et la division par 4 en fait une moyenne : exactement le gradient d'un batch de 64, pour la mémoire d'un batch de 16.

QCM 3 : niveau difficile

Calculs en plusieurs étapes, bugs PyTorch à repérer, pièges du cours et questions de réflexion (expliquer, diagnostiquer, choisir).

Grille de réponses

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
21	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**

Question 1 S2 · Gradient descent

Deux exemples : $x^{(1)} = (1, 2)$, $y^{(1)} = 1$ et $x^{(2)} = (2, 0)$, $y^{(2)} = 0$. Départ $w = (0, 0)$, $b = 0$, $\eta = 1$. Que valent w et b après un pas de gradient descent sur le coût J ?

- **A.** w = (0.25, -0.5) et b = 0
- **B.** w = (-0.5, 1) et b = 0
- **C.** w = (-0.25, 0.5) et b = -0.5
- **D.** w = (-0.25, 0.5) et b = 0

Question 2 S2 · Autograd

Qu'affiche ce code ?

```
w = torch.tensor([1.0, 1.0], requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
x, y = torch.tensor([1.0, -1.0]), 0.0
a = torch.sigmoid(w @ x + b)
loss = -(y * torch.log(a) + (1 - y) * torch.log(1 - a))
loss.backward()
print(w.grad, b.grad)
```

- **A.** tensor([-0.5000, 0.5000]) tensor(-0.5000)
- **B.** tensor([0.1250, -0.1250]) tensor(0.1250)
- **C.** tensor([0., 0.]) tensor(0.)
- **D.** tensor([0.5000, -0.5000]) tensor(0.5000)

Question 3 S2 · PyTorch

Cette boucle manuelle (sans optimiseur) apprend de plus en plus mal. Quelle étape manque ?

```
for it in range(200):
    A = torch.sigmoid(X_t @ w_t + b_t)
    loss = -(y_t * torch.log(A) + (1 - y_t) * torch.log(1 - A)).mean()
    loss.backward()
    with torch.no_grad():
        w_t -= eta * w_t.grad
        b_t -= eta * b_t.grad
```

- **A.** Sortir la mise à jour du bloc `torch.no_grad()`
- **B.** Remettre `w_t.grad` et `b_t.grad` à zéro
- **C.** Mettre `requires_grad=False` sur `w_t` et `b_t`
- **D.** Appeler `loss.backward()` une seconde fois

Question 4 S2 · Why not MSE

Exemple de classe y = 1 très mal prédit : a = 0.01. Que vaut $\partial\mathcal{L}/\partial z$ avec la MSE ($\mathcal{L} = (a - y)^2$), puis avec la log loss ?

- **A.** MSE ≈ -0.0196 ; log loss = -0.99
- **B.** MSE = -0.99 ; log loss ≈ -0.0196
- **C.** MSE ≈ -1.98 ; log loss = -0.99
- **D.** MSE = -0.99 ; log loss = -0.99

Question 5 S2 · Log loss

Pour construire la log loss, le cours prend le log de la vraisemblance $L = \Pi_i P(y^{(i)} \mid x^{(i)})$, puis minimise $-(1/m) \log L$. Pourquoi le log ?

- **A.** Il rend la loss exactement nulle sur chaque exemple bien classé du dataset
- **B.** Il transforme la sigmoïde en une fonction linéaire des poids w
- **C.** Il double le gradient de chaque exemple mal classé du dataset
- **D.** Il est croissant, et un produit de m nombres < 1 s'arrondirait à 0

Question 6 S3 · Backpropagation

Réseau 2-2-1 sigmoïde avec log loss : x = (1, 2), y = 0, $W^{[1]} = [[1, -1], [0.5, 0.5]]$, $b^{[1]} = (1, -1.5)$, $W^{[2]} = (1, -1)$, $b^{[2]} = 0$. Que vaut $dW^{[1]}$?

- **A.** [[-0.125, -0.25], [0.125, 0.25]]
- **B.** [[0.5, 1], [-0.5, -1]]
- **C.** [[0.25, 0.25], [0.25, 0.25]]
- **D.** [[0.125, 0.25], [-0.125, -0.25]]

Question 7 S3 · Autograd

Qu'affiche ce code ?

```
w = torch.tensor(3.0, requires_grad=True)
for _ in range(2):
    loss = w ** 2
    loss.backward()
print(w.grad)
```

- **A.** tensor(6.)
- **B.** tensor(9.)
- **C.** tensor(12.)
- **D.** tensor(18.)

Question 8 S3 · Non-linearity

Le réseau XOR du cours, SANS activation : $W^{[1]} = [[1, 1], [-1, -1]]$, $b^{[1]} = (-0.5, 1.5)$, $W^{[2]} = (1, 1)$, $b^{[2]} = -1.5$. Que valent $W' = W^{[2]}W^{[1]}$ et $b' = W^{[2]}b^{[1]} + b^{[2]}$?

- **A.** W' = (0, 0) et b' = -0.5 : sortie constante
- **B.** W' = (1, 1) et b' = -1.5 : on retrouve AND
- **C.** W' = (2, 2) et b' = 0 : on retrouve OR
- **D.** W' = (0, 0) et b' = 0 : sortie toujours nulle

Question 9 S3 · Hidden layer

Pourquoi le réseau XOR (activation marche) fonctionne-t-il ? Dans l'espace caché (h_1, h_2), les 4 entrées deviennent :

- **A.** 3 points distincts, séparables par une droite
- **B.** 4 points distincts, qui restent impossibles à séparer
- **C.** un seul point, quelle que soit l'entrée
- **D.** 4 points placés sur un cercle autour de 0

Question 10 S3 · Non-convexity

Entraînement NumPy du XOR (2-2-1, $\eta = 2$, 3000 itérations) : avec la seed 2, le coût passe de 0.753 à 0.0016 ; avec la seed 0, il stagne vers 0.35. Pourquoi ?

- **A.** La couche cachée rend le coût non convexe : minimum local
- **B.** La seed 0 donne un learning rate effectif beaucoup plus petit
- **C.** XOR n'est pas séparable, même avec une couche cachée
- **D.** Il faudrait des labels one-hot avec la seed 0

Question 11 S4 · Losses

Quel est le problème de ce code ?

```
model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(),
                      nn.Linear(128, 10), nn.Softmax(dim=1))
loss_fn = nn.CrossEntropyLoss()
```

- **A.** La softmax est appliquée deux fois
- **B.** Il faut des labels one-hot avec cette loss
- **C.** `nn.Softmax` doit précéder la dernière couche
- **D.** Aucun : c'est la façon recommandée de faire

Question 12 S4 · Losses

`loss_fn = nn.CrossEntropyLoss()` et `logits` a la forme (128, 10). Quelle forme et quel type doit avoir `yb` ?

- **A.** (128, 10) en float32 : un one-hot par image
- **B.** (128,) en int64 : un numéro de classe par image
- **C.** (10,) en int64 : un numéro par classe
- **D.** (128, 1) en float32 : une probabilité par image

Question 13 S4 · Optimisers

On entraîne `model_a`, puis on crée `model_b`, mais on réutilise `optimizer = torch.optim.SGD(model_a.parameters(), lr=0.2)`. Que se passe-t-il pendant l'entraînement de `model_b` ?

- **A.** Ses poids ne bougent pas : l'optimiseur pointe vers `model_a`
- **B.** Il s'entraîne normalement : l'optimiseur est partagé par tous les modèles
- **C.** Il s'entraîne deux fois plus vite grâce à `model_a`
- **D.** PyTorch copie les poids de `model_a` dans `model_b`

Question 14 S4 · eval() et no_grad()

Laquelle de ces affirmations est vraie ?

- **A.** `no_grad()` change dropout et batch norm ; `eval()` coupe le graphe
- **B.** `no_grad()` coupe le graphe ; `eval()` change dropout et batch norm
- **C.** `model.eval()` et `torch.no_grad()` font exactement la même chose
- **D.** `model.eval()` met `requires_grad = False` sur tous les poids

Question 15 S4 · DataLoader

TP05 : 50 000 images, `batch_size=128`, `shuffle=True`. Combien d'images contient le dernier batch de chaque epoch ?

- **A.** 128
- **B.** 80
- **C.** 48
- **D.** 0

Question 16 S4 · DataLoader

Pourquoi `shuffle=True` pour le `train_loader`, mais pas pour les loaders de validation et de test ?

- **A.** L'ordre ne compte que pour les mises à jour d'entraînement
- **B.** Mélanger la validation fausserait le calcul de l'accuracy
- **C.** Le test doit garder son ordre à cause du gradient clipping
- **D.** `shuffle=True` est interdit avec `batch_size=1000`

Question 17 S5 · Backpropagation

Petit réseau ReLU : $z_1 = w_1x + b_1$, $a_1 = \text{ReLU}(z_1)$, $z_2 = w_2a_1 + b_2$, $a_2 = \text{ReLU}(z_2)$, $\hat{y} = w_3a_2$, $\mathcal{L} = ½(\hat{y} - y)^2$, avec $x = 2$, $y = 3$, $w_1 = 0.5$, $b_1 = 1$, $w_2 = -1$, $b_2 = 3$, $w_3 = 2$. Que vaut $\partial\mathcal{L}/\partial w_1$?

- **A.** -4
- **B.** 2
- **C.** 4
- **D.** 0

Question 18 S5 · Dead ReLU

Même réseau, mais avec $b_2 = 0$ au lieu de 3. Que valent $\hat{y}$, $\mathcal{L}$ et les gradients ?

- **A.** $\hat{y} = 0$, $\mathcal{L} = 4.5$ et tous les gradients sont nuls
- **B.** $\hat{y} = 0$, $\mathcal{L} = 4.5$ et les gradients sont non nuls
- **C.** $\hat{y} = 2$, $\mathcal{L} = 0.5$ et tous les gradients sont nuls
- **D.** $\hat{y} = -4$, $\mathcal{L} = 24.5$ et les gradients sont non nuls

Question 19 S5 · Vanishing gradient

Chaîne de couches sigmoïdes : pourquoi prendre de grands poids ($w = 4$, pour que $4 \times 1/4 = 1$) ne résout-il pas le vanishing gradient ?

- **A.** Parce que de grands poids rendent le réseau entièrement linéaire
- **B.** $4\sigma'(z) = 1$ seulement en $z = 0$, et de grands poids font saturer
- **C.** Parce que la sortie applique déjà une ReLU
- **D.** Parce que le gradient devient alors exactement nul

Question 20 S5 · Activations

TP05, Partie C (6 couches de 128, 5 epochs) : tanh atteint 0.868 de val accuracy alors que la sigmoïde reste à 0.097. Pourquoi tanh s'en sort-elle à cette profondeur ?

- **A.** tanh ne sature jamais, même pour de très grands $|z|$
- **B.** tanh est linéaire autour de 0, donc son gradient vaut toujours 1
- **C.** tanh est une ReLU lissée, de dérivée 1 pour $z > 0$
- **D.** $\tanh'(0) = 1$, quatre fois $\sigma'(0)$, et tanh est centrée sur 0

Question 21 S6 · Weight decay

Weight decay avec η = 0.1 et λ = 0.1, et un gradient des données nul pour un poids donné. Après combien de pas ce poids est-il divisé par 2 environ ?

- **A.** Environ 14 pas
- **B.** Environ 7 pas
- **C.** Environ 693 pas
- **D.** Environ 69 pas

Question 22 S6 · Dropout

Un modèle contient `nn.Dropout(0.5)` et vient d'être entraîné (il est donc en mode `train()`). On l'évalue ainsi :

```
with torch.no_grad():
    logits = model(X_val)
    acc = (logits.argmax(dim=1) == y_val).float().mean()
```

Que se passe-t-il ?

- **A.** Rien : `torch.no_grad()` coupe aussi le dropout et la batch norm
- **B.** Le dropout reste actif : accuracy bruitée et plus basse
- **C.** Les poids sont mis à jour pendant ce calcul
- **D.** Les sorties du dropout sont divisées deux fois par (1 - p)

Question 23 S6 · Batch norm

Une couche `nn.BatchNorm1d(1)` neuve (running_mean = 0, running_var = 1, momentum = 0.1) voit un seul batch d'entraînement z = (2, 2, 6, 6). Que valent ensuite ses statistiques glissantes ?

- **A.** running_mean = 4 ; running_var = 4
- **B.** running_mean = 0.4 ; running_var = 1.3
- **C.** running_mean = 3.6 ; running_var ≈ 4.9
- **D.** running_mean = 0.4 ; running_var ≈ 1.43

Question 24 S6 · Adam

Au tout premier pas (t = 1), avec les corrections de biais, de combien Adam (η = 0.01) déplace-t-il un poids dont le gradient vaut g = 250 (ε négligé) ?

- **A.** De 2.5, soit η × g
- **B.** De 0.01, soit η × signe(g)
- **C.** De 0.25, soit 0.1 × η × g
- **D.** De 0, car m et s partent de 0

Question 25 S6 · Momentum

Momentum, convention PyTorch ($v \leftarrow \beta v + g$, puis $w \leftarrow w - \eta v$), sur $J(w) = w^2$ avec w = 2, v = 0, η = 0.1 et β = 0.5. Que vaut w après deux pas ?

- **A.** 1.28
- **B.** 1.20
- **C.** 0.88
- **D.** 1.08

Question 26 S6 · Diagnosis

Avec cette boucle, la loss passe à 1408 dès la première epoch, puis devient `nan` :

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
for xb, yb in train_loader:
    loss = loss_fn(model(xb), yb)
    loss.backward()
    optimizer.step()
```

Quelle est la cause ?

- **A.** Le learning rate 0.05 est trop grand pour SGD
- **B.** `loss_fn` attend des probabilités, pas des logits
- **C.** Les gradients s'additionnent d'un pas à l'autre
- **D.** Le DataLoader ne mélange pas les données

Question 27 S6 · Regularisation

Tableau du TP06 (val_acc, gap) : baseline (0.815, 0.164), weight decay (0.789, 0.096), dropout (0.768, 0.068), early stopping (0.818, 0.082). Quelle conclusion est conforme au cours ?

- **A.** Le dropout est le meilleur réglage car son gap est le plus petit
- **B.** Le weight decay améliore la val_acc par rapport à la baseline
- **C.** L'early stopping est la méthode la plus coûteuse en calcul
- **D.** Un petit gap n'est pas un but en soi : regarder aussi val_acc

Question 28 S6 · Early stopping

Dans sa boucle d'early stopping, un étudiant écrit `best_state = model.state_dict()` (sans `copy.deepcopy`). Quel est le problème ?

- **A.** `state_dict()` ne contient que les gradients, pas les poids
- **B.** `load_state_dict` refuse un dictionnaire non copié
- **C.** `best_state` suit l'entraînement : il contient des références
- **D.** Aucun : `state_dict()` renvoie déjà une copie figée

Question 29 S7 · Parameters

Combien de paramètres a ce CNN (entrée 1 × 28 × 28 ; forward : conv1, ReLU, pool, conv2, ReLU, pool, flatten, fc) ?

```
self.conv1 = nn.Conv2d(1, 16, 3, padding=1)
self.conv2 = nn.Conv2d(16, 32, 3, padding=1)
self.pool = nn.MaxPool2d(2)
self.fc = nn.Linear(32 * 7 * 7, 10)
```

- **A.** 20 490
- **B.** 20 432
- **C.** 67 530
- **D.** 16 170

Question 30 S7 · Shapes

Pour une entrée de forme (1, 1, 28, 28), quelle est la taille du vecteur en sortie de `nn.Flatten()` ?

```
net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1),
    nn.MaxPool2d(2),
    nn.Conv2d(8, 16, 3),
    nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, stride=2, padding=1),
    nn.Flatten(),
)
```

A. 1 152

B. 512

C. 288

D. 800

Question 31 S7 · Transfer learning

Après ce code, quels paramètres seront réellement entraînés ?

```
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(512, 10)
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

A. Seulement la nouvelle tête `fc` (5 130 paramètres)

B. Aucun : la boucle a aussi gelé la nouvelle `fc`

C. Tous les paramètres du ResNet-18, y compris la tête

D. Seulement `layer4` et la nouvelle tête `fc`

Question 32 S7 · ResNet

Bloc résiduel y = F(x) + x (on ignore la ReLU finale). Que vaut ∂y/∂x, et pourquoi est-ce important ?

A. ∂F/∂x × x : le gradient est multiplié par x

B. ∂F/∂x seulement, car x est une constante du bloc

C. ∂F/∂x + I : l'identité garde le gradient vivant

D. I seulement : F n'a pas de gradient

Question 33 S7 · Translation

Laquelle de ces affirmations sur les translations est correcte ?

A. Une convolution rend la sortie identique quel que soit le décalage

B. Un réseau dense réutilise ses poids à toutes les positions

C. Le max pooling donne une invariance locale aux petits décalages

D. Un CNN avec une dernière couche dense est totalement invariant

Question 34 S7 · Transfer learning

Projet : classer des peintures, très différentes des photos d'ImageNet, avec beaucoup d'images. Quelle stratégie le tableau du cours recommande-t-il ?

A. Tout geler et n'entraîner que la nouvelle tête

B. Dégeler seulement conv1, avec un grand learning rate

C. Tout fine-tuner, avec un petit learning rate

D. Repartir de zéro, sans aucun poids pré-entraîné

Question 35 S8 · Attention

Attention : s = (0, 1), h_1 = (1, 0), h_2 = (0, 2), h_3 = (1, 1). Que vaut le vecteur de contexte c (arrondi) ?

- **A.** (0.667, 1.000)
- **B.** (1, 5)
- **C.** (0.335, 1.575)
- **D.** (0.335, 0.910)

Question 36 S8 · LSTM

Pourquoi le gradient traverse-t-il bien mieux le cell state d'une LSTM que l'état caché d'un RNN simple ?

- **A.** Le cell state est remis à zéro à chaque pas de temps
- **B.** Sur ce chemin, $\partial c_t/\partial c_{t-1} = f_t$: ni tanh' ni W_{hh}
- **C.** La LSTM n'utilise pas la backpropagation through time
- **D.** Les gates sont des ReLU, de dérivée toujours égale à 1

Question 37 S8 · RNN

Quelles sont les formes de `out` et `h_n` ?

```
rnn = nn.RNN(input_size=3, hidden_size=4, batch_first=True)
x = torch.randn(2, 6, 3)
out, h_n = rnn(x)
```

- **A.** out : (2, 6, 3) ; h_n : (2, 4)
- **B.** out : (6, 2, 4) ; h_n : (1, 6, 4)
- **C.** out : (2, 4) ; h_n : (2, 6, 4)
- **D.** out : (2, 6, 4) ; h_n : (1, 2, 4)

Question 38 S8 · Attention

Dans l'exercice d'attention du cours (scores e = (2, 1, 3)), on multiplie tous les scores par 10. Que deviennent les poids α ?

- **A.** Inchangés : la softmax ne voit que l'ordre
- **B.** Presque one-hot : α ≈ (0, 0, 1)
- **C.** Uniformes : α = (1/3, 1/3, 1/3)
- **D.** Multipliés par 10 : ils somment à 10

Question 39 S8 · Production checklist

Un camarade annonce 87.4 % avec une nouvelle augmentation contre 87.1 % sans : une exécution chacune, mesurées sur le test set après avoir essayé 10 variantes. Que lui demander, d'après la checklist ?

- **A.** Rien : 0.3 point d'écart sur le test suffit pour conclure
- **B.** D'entraîner plus longtemps pour creuser l'écart entre les deux
- **C.** Plusieurs seeds (moyenne, écart) et un choix sur validation
- **D.** De passer de SGD à Adam pour confirmer que le gain est réel

Question 40 Synthèse du cours

La slide « The whole course in one slide » conclut : « Every fix (ReLU, He, batch norm, ResNet, LSTM) is about... ». Quelle est la fin de la phrase ?

- **A.** reducing the number of parameters
- **B.** letting the gradient flow
- **C.** making the loss convex
- **D.** removing the need for data

Corrigé : QCM 3 : niveau difficile

Compte 1 point par bonne réponse. Repère : 28/40 ou plus, tu es prêt pour l'examen ; moins de 20/40, reprends les corrigés ligne par ligne, puis refais ce QCM dans quelques jours.

N°	Réponse	Explication
1	**D**	a = (0.5, 0.5), donc A - y = (-0.5, 0.5). dw = ½[(-0.5)(1, 2) + 0.5(2, 0)] = ½(0.5, -1) = (0.25, -0.5) et db = ½(-0.5 + 0.5) = 0. w ← w - η dw = (-0.25, 0.5). (-0.5, 1) oublie le 1/m ; (0.25, -0.5) va dans le sens du gradient.
2	**D**	z = 1 - 1 + 0 = 0, donc a = 0.5. Gradient = (a - y) x = 0.5 × (1, -1) = (0.5, -0.5) et $\partial\mathcal{L}/\partial b$ = a - y = 0.5. (0.125, ...) garde à tort le facteur a(1 - a) ; z = 0 ne veut pas dire gradient nul.
3	**B**	« PyTorch adds new gradients to the old ones » : sans `w_t.grad.zero_()` et `b_t.grad.zero_()`, les gradients s'accumulent d'une itération à l'autre. La mise à jour doit, elle, rester dans `torch.no_grad()` : on modifie les paramètres, cela ne doit pas être enregistré.
4	**A**	MSE : 2(a - y) a(1 - a) = 2 × (-0.99) × 0.01 × 0.99 ≈ -0.0196, presque aucune correction alors que le modèle se trompe fortement. Log loss : a - y = -0.99, sans facteur a(1 - a) pour « tuer » le gradient (et elle est convexe).
5	**D**	Le log est croissant : même maximum. Il transforme le produit en somme, alors qu'un produit de m nombres inférieurs à 1 « underflows to 0 ». Diviser par m donne une moyenne indépendante de la taille du dataset.
6	**D**	Forward : $z^{[1]}$ = (0, 0), $a^{[1]}$ = (0.5, 0.5), $z^{[2]}$ = 0, $a^{[2]}$ = 0.5. Backward : $dz^{[2]} = a^{[2]} - y = 0.5$; $da^{[1]} = W^{[2]T} dz^{[2]}$ = (0.5, -0.5) ; $dz^{[1]} = da^{[1]} \odot a^{[1]}(1 - a^{[1]})$ = (0.125, -0.125) ; $dW^{[1]} = dz^{[1]} x^T$. Avec y = 1 (exercice du cours), tous les signes s'inversent.
7	**C**	$\partial(w^2)/\partial w = 2w = 6$. Mais `backward()` AJOUTE au `.grad` existant : après deux appels sans remise à zéro, w.grad = 6 + 6 = 12. C'est pour cela que la boucle d'entraînement appelle `optimizer.zero_grad()`.
8	**A**	W' = (1 - 1, 1 - 1) = (0, 0) et b' = (-0.5 + 1.5) - 1.5 = -0.5 : le réseau renvoie -0.5 pour toutes les entrées. Sans activation non linéaire entre les couches, l'empilement se réduit à une seule couche linéaire.
9	**A**	(0, 0) → (0, 1) ; (0, 1) et (1, 0) → (1, 1) ; (1, 1) → (1, 0). La droite $h_1 + h_2 - 1.5 = 0$ sépare les classes. « A hidden layer learns new features in which the problem becomes easy for the last neuron. »
10	**A**	Pour la régression logistique, la log loss est convexe : gradient descent atteint le minimum. Avec une couche cachée, le coût n'est plus convexe : selon l'initialisation (la seed), on peut rester bloqué dans un minimum local.
11	**A**	nn.CrossEntropyLoss applique déjà la softmax puis le -log sur des logits : la softmax du modèle s'ajoute à la sienne et écrase les gradients. Le modèle doit finir par nn.Linear(128, 10), sans activation.
12	**B**	CrossEntropyLoss attend des numéros de classe (int64), pas des vecteurs one-hot : yb a la forme (batch,) = (128,). C'est pour cela que les labels FashionMNIST restent en int64 alors que les images passent en float32.
13	**A**	Un optimiseur est « lié aux paramètres d'un modèle précis » : `step()` met à jour les paramètres de model_a, pas ceux de model_b. Il faut un nouvel optimiseur par réseau (le TP06 les construit avec des lambdas pour cette raison).
14	**B**	`torch.no_grad()` désactive l'enregistrement du graphe (pas de gradients, moins de mémoire). `model.eval()` change le comportement de certaines couches : dropout devient l'identité, batch norm utilise ses running statistics. Pour évaluer, on utilise les deux.
15	**B**	390 batchs complets font 390 × 128 = 49 920 images ; le 391e contient les 50 000 - 49 920 = 80 restantes. D'où `running += loss.item() * len(yb)` pour une moyenne juste.
16	**A**	À l'entraînement, un ordre différent à chaque epoch évite que l'ordre des exemples biaise les mises à jour. En validation et en test, il n'y a aucune mise à jour : l'ordre ne change ni la loss moyenne ni l'accuracy.

N°	Réponse	Explication
17	**C**	Forward : $\hat{y} = 2$, donc $\partial\mathcal{L}/\partial\hat{y} = -1$. Puis $\partial\mathcal{L}/\partial z_2 = -1 \times w_3 \times \text{ReLU}'(z_2) = -2$; $\partial\mathcal{L}/\partial z_1 = -2 \times w_2 \times \text{ReLU}'(z_1) = 2$; $\partial\mathcal{L}/\partial w_1 = 2 \times x = 4$. Soit (-1)(2)(1)(-1)(1)(2) = 4.
18	**A**	$z_2 = -1 \times 2 + 0 = -2 < 0$, donc $a_2 = 0$ et $\hat{y} = 0$: $\mathcal{L} = ½(0 - 3)^2 = 4.5$. Mais $\text{ReLU}'(z_2) = 0$ coupe la chaîne : TOUS les gradients valent exactement 0 malgré une grosse loss. C'est le mécanisme de la dead ReLU.
19	**B**	$4\sigma'(z)$ vaut 1 seulement en z = 0 (0.94 en 0.5, 0.79 en 1, 0.42 en 2, 0.03 en 5). Or de grands poids poussent z loin de 0, là où la sigmoïde sature. « Le vrai coupable, c'est la sigmoïde » : la solution est ReLU.
20	**D**	$\tanh(z) = 2\sigma(2z) - 1$: sa dérivée atteint 1 en z = 0 (contre 1/4 pour σ) et elle est centrée sur 0. Elle sature quand même : avec un réseau beaucoup plus profond, tanh échouerait aussi. Ratio des gradients première/dernière couche : 2.0 pour tanh, 4.7×10^{-5} pour la sigmoïde.
21	**D**	Chaque pas multiplie le poids par $1 - \eta\lambda = 0.99$. On cherche $0.99^k = 0.5$: $k = \ln 0.5 / \ln 0.99 \approx 69$. Règle du cours : $k \approx \ln 2 / (\eta\lambda) = 0.693 / 0.01 \approx 69$. (14 pas correspond à l'exemple du cours, où $\eta\lambda = 0.05$.)
22	**B**	`torch.no_grad()` arrête seulement l'enregistrement du graphe pour autograd. C'est `model.eval()` qui transforme le dropout en identité (et fait utiliser les running statistics à la batch norm). Le cours appelle ça « the classic bug ».
23	**D**	$\mu_{run} \leftarrow 0.9 \times 0 + 0.1 \times 4 = 0.4$. Pour la variance, PyTorch utilise la version non biaisée $m/(m - 1) \times \sigma_B^2 = 4/3 \times 4 \approx 5.33$: $0.9 \times 1 + 0.1 \times 5.33 \approx 1.43$. 1.3 serait obtenu avec la variance biaisée 4 ; 3.6 et 4.9 inversent les poids 0.9 et 0.1.
24	**B**	À t = 1 : $\hat{m} = (1 - \beta_1)\, g / (1 - \beta_1) = g$ et $\hat{s} = (1 - \beta_2)\, g^2 / (1 - \beta_2) = g^2$. Le pas vaut $\eta \times g / \lvert g \rvert = \eta \times \text{signe}(g)$: le premier pas a une taille η quelle que soit la taille du gradient.
25	**D**	Pas 1 : g = 4, v = 4, w = 2 - 0.4 = 1.6. Pas 2 : g = 3.2, v = 0.5 × 4 + 3.2 = 5.2, w = 1.6 - 0.52 = 1.08. La SGD simple donne 1.28 ; 1.20 utilise l'ancien v ; 0.88 prend β = 1.
26	**C**	Il manque `optimizer.zero_grad()` : `loss.backward()` AJOUTE au `.grad` existant, le gradient grossit à chaque pas (jusqu'à 32 fois à la fin de l'epoch). Même symptôme qu'un η trop grand, autre cause : relire la boucle avant de toucher à η.
27	**D**	Le dropout a le plus petit gap mais la val_acc la plus basse : il a réduit l'écart en freinant l'apprentissage (λ et p se règlent sur la validation). L'early stopping est gratuit et garde la meilleure epoch. « The first fix is always more data. »
28	**C**	`state_dict()` renvoie des références vers les tenseurs vivants, qui continuent de changer : sans `deepcopy`, la « sauvegarde » suit les poids actuels et on ne revient jamais à la meilleure epoch.
29	**A**	conv1 : 16 × (1 × 9 + 1) = 160. conv2 : 32 × (16 × 9 + 1) = 4 640. fc : 1 568 × 10 + 10 = 15 690. Total 20 490, contre 235 146 pour le réseau dense de la session 5 (inductive bias : locality et weight sharing). Le pooling n'a aucun paramètre.
30	**C**	28 → conv (p = 1) 28 → pool 14 → conv 3×3 sans padding 12 → pool 6 → conv stride 2, p = 1 : $\lfloor(6 + 2 - 3)/2\rfloor + 1 = 3$. Sortie 32 × 3 × 3 = 288. (1 152 = 32 × 6 × 6 oublie le stride.)
31	**A**	La boucle gèle les paramètres existants. La nouvelle couche `nn.Linear(512, 10)` est créée APRÈS : ses paramètres ont `requires_grad = True` par défaut, et l'optimiseur ne reçoit qu'elle. C'est exactement le code du cours.
32	**C**	La dérivée d'une somme est la somme des dérivées : $\partial y/\partial x = \partial F/\partial x + I$. En empilant les blocs, le gradient qui arrive au bloc l contient toujours $\partial J/\partial x_L$ lui-même, sans produit de petits facteurs : il ne peut pas disparaître (dans le cours : 4×10^{-14} sans skip contre 6×10^{-2} avec, au bloc 1).
33	**C**	Pooling : « if a feature moves by one pixel inside its window, the maximum does not change ». La convolution, elle, est équivariante : décaler l'entrée décale la feature map. Le CNN du TP07 n'est pas totalement invariant (dernière couche dense, objets toujours centrés) : la data augmentation corrigerait le reste.
34	**C**	Peu d'images proches des photos : tout geler et entraîner la tête. Peu à moyen, un peu différentes : dégeler aussi le dernier bloc avec un petit lr. Beaucoup d'images ou très différentes (radios, peintures) : tout fine-tuner avec un petit lr. Et toujours préparer les entrées comme au pré-entraînement (moyenne et std ImageNet).

N°	Réponse	Explication
35	**C**	Scores e = (0, 2, 1). Softmax : $e^0 = 1$, $e^2 \approx 7.389$, $e^1 \approx 2.718$, somme ≈ 11.107, donc α ≈ (0.090, 0.665, 0.245). c = 0.090 × (1, 0) + 0.665 × (0, 2) + 0.245 × (1, 1) ≈ (0.335, 1.575). (0.667, 1.000) prend α = 1/3 ; (1, 5) oublie la softmax.
36	**B**	$c_t = f_t \odot c_{t-1} + i_t \odot g_t$ est une mise à jour additive : en remontant d'un pas, on multiplie seulement par f_t. Si $f_t \approx 1$, mémoire et gradient traversent de nombreux pas. Même idée que la skip connection de ResNet : un chemin identité pour le gradient.
37	**D**	Avec batch_first=True, l'entrée est (batch, time, features) = (2, 6, 3). `out` contient tous les états cachés : (batch, time, hidden) = (2, 6, 4). `h_n` est le dernier, et garde les couches en premier même avec batch_first : (1, 2, 4).
38	**B**	softmax(20, 10, 30) ≈ (4.5×10^{-5}, 2×10^{-9}, 0.99995) : de grands écarts de scores rendent la softmax presque one-hot. C'est pour garder les scores dans une plage raisonnable que le transformer divise QK^T par √d.
39	**C**	Une seed = un tirage : un gain de 0.3 % est souvent du bruit, il faut la moyenne et l'écart sur plusieurs seeds. Et choisir parmi 10 variantes sur le test en fait une deuxième validation : le test ne sert qu'une fois, à la fin.
40	**B**	« One thread: a model, a loss, its gradient, a better architecture when the previous one hits a wall. Every fix (ReLU, He, batch norm, ResNet, LSTM) is about letting the gradient flow. »